VIETNAM GENERAL CONFEDERATION OF LABOR
TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY

HUYNH NGOC NHI – 523H0066
NGUYEN THI ANH THU – 523H0100

FINAL REPORT

KITTACHAT
REAL-TIME CHAT & VIDEO CALL PLATFORM

FULL-STACK SOFTWARE DEVELOPMENT

HO CHI MINH CITY, 2026

VIETNAM GENERAL CONFEDERATION OF LABOR
TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY

HUYNH NGOC NHI – 523H0066
NGUYEN THI ANH THU – 523H0100

FINAL REPORT

KITTACHAT
REAL-TIME CHAT & VIDEO CALL PLATFORM

FULL-STACK SOFTWARE DEVELOPMENT

Advised by
MSc. Vu Dinh Hong

HO CHI MINH CITY, 2026

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to Ton Duc Thang University and the Faculty of Information Technology for providing a high-quality academic program, especially the **Full Stack Software Development** course within the Software Engineering major. This course has given us valuable opportunities to learn, practice, and apply modern technologies in developing complete applications from frontend to backend.

We are especially grateful **MSc. Vu Dinh Hong**, the course instructor, for his dedication, insightful guidance, and continuous support throughout our learning journey and the completion of this project.

As this is our first time conducting a project for this course, our report may still have some shortcomings. We sincerely look forward to receiving constructive feedback so that we can further improve our knowledge and skills in the future.

We wish you good health, happiness, and continued success.

Sincerely,

Ho Chi Minh city, 20th April 2026

Author

(Signature and full name)

Huỳnh Ngoc Nhi

Nguyen Thi Anh Thu

DECLARATION OF AUTHORSHIP

We hereby declare that this is our own project and is guided by MSc. Vu Dinh Hong; The content research and results contained herein are central and have not been published in any form before. The data in the tables for analysis, comments and evaluation are collected by the main author from different sources, which are clearly stated in the reference section.

In addition, the project also uses some comments, assessments as well as data of other authors, other organizations with citations and annotated sources.

If something wrong happens, we'll take full responsibility for the content of my project. Ton Duc Thang University is not related to the infringing rights, the copyrights that We give during the implementation process (if any).

Ho Chi Minh city, 10th April 2026

Author

(Signature and full name)

Huynh Ngoc Nhi

Nguyen Thi Anh Thu

REAL-TIME CHAT & VIDEO CALL PLATFORM

ABSTRACT

In the era of digital transformation, real-time communication platforms such as Zalo, Discord, and Slack have become essential tools for both personal and professional interactions. However, developing a scalable and high-performance real-time communication system remains a significant challenge.

This project presents a **Real-Time Chat & Video Call Platform**, a web-based application that supports instant messaging, multimedia sharing, group chat, and peer-to-peer audio/video calls. The system focuses on solving key issues such as scalability, low latency, and efficient data processing.

The platform is built using **ReactJS** for the frontend and **Node.js** with **Socket.io** for real-time communication. **MongoDB** is used for message storage, while **Redis Pub/Sub** enables synchronization across multiple server instances. **WebRTC** is applied for direct peer-to-peer audio and video communication, reducing server load and improving performance.

The system is deployed using **Docker** and **Docker Compose**, with **Nginx** acting as a reverse proxy for load balancing. The final result is a stable and scalable real-time communication platform, demonstrating practical knowledge in full-stack development and distributed system design.

TABLE OF CONTENT

LIST OF FIGURES

LIST OF TABLES

ABBREVIATIONS

API	Application Programming Interface
CSS	Cascading Style Sheets
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
ICE	Interactive Connectivity Establishment
IDE	Integrated Development Environment
JSX	JavaScript XML
RESTful APIs	Representational State Transfer APIs
SDP	Session Description Protocol
TCP	Transmission Control Protocol
UI	User Interface
WebRTC	Web Real-Time Communication

CHAPTER 1. INTRODUCTION

1.1 Background and Motivation

In recent years, the demand for real-time online communication has grown rapidly, especially in areas such as remote work, online learning, and digital collaboration. Platforms such as Zalo, Discord, and Slack have become essential tools by enabling instant messaging, group communication, and real-time audio/video interaction.

However, building a system that can efficiently handle real-time communication while maintaining high performance and scalability remains a significant challenge. Traditional request-response web communication models are not well-suited for continuous real-time interactions. As the number of concurrent users increases, systems often face issues such as high latency, unstable performance, and server overload. In particular, ensuring scalability across multiple server instances while maintaining consistent real-time data synchronization is a critical and complex problem in distributed systems.

To address these challenges, this project aims to develop a **Real-Time Chat & Video Call Platform** named Kittachat that supports instant messaging, file sharing, group chat, and peer-to-peer audio or video communication. The project focuses not only on implementing core communication features but also on optimizing system performance and scalability by leveraging modern technologies such as WebSocket for real-time data exchange, WebRTC for peer-to-peer media streaming, Redis for caching and Pub/Sub messaging, and containerized deployment using Docker. These technologies collectively contribute to building a stable, efficient, and scalable real-time communication system.

Figure 1 *Internal Communication*

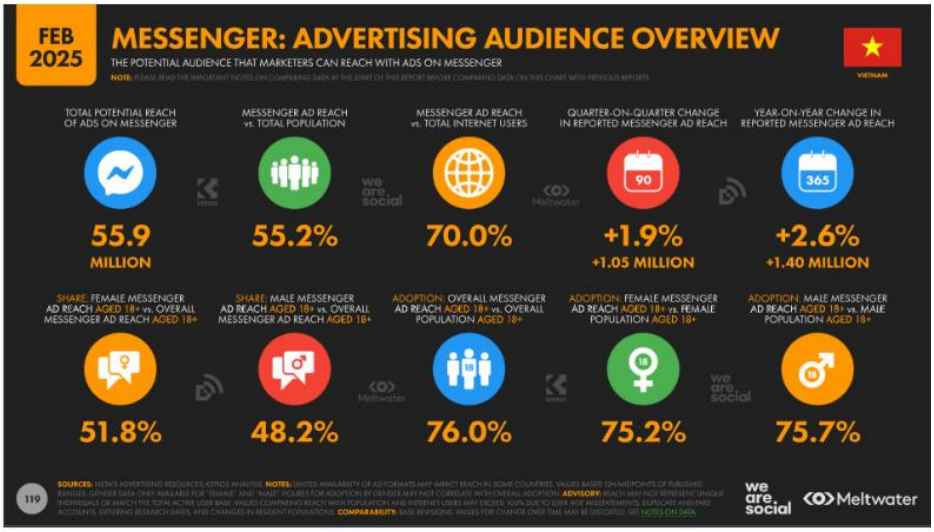

Figure 2 *Statistics on Messenger users for communication in Vietnam in 2025*

Figure 3 *Monthly active users of the Zalo application in Vietnam up to Q3 2025*

1.2 Objectives of the project

The primary objective of this project is to design and develop a real-time communication platform that supports instant messaging and audio or video calling functionalities in a web-based environment. The system aims to provide a seamless and responsive user experience while ensuring high performance and scalability under concurrent user loads.

Specifically, the project focuses on the following objectives:

- To build a real-time messaging system that enables users to send and receive messages instantly without page reload, using WebSocket-based communication.
- To implement user management features, including authentication, friend management, and user search functionality.
- To develop group chat capabilities, allowing multiple users to communicate within shared chat rooms.
- To support multimedia messaging, including image and file sharing within conversations.
- To integrate peer-to-peer audio and video calling using WebRTC, ensuring low-latency media transmission between users.
- To design and implement a scalable backend architecture capable of handling multiple concurrent connections, utilizing technologies such as Redis for caching and Pub/Sub mechanisms.
- To deploy the system using containerization tools such as Docker, ensuring ease of setup, portability, and consistency across environments.

In addition to functional requirements, the project also aims to address non-functional aspects such as system performance, reliability, and scalability. By applying modern web technologies and distributed system principles, the system is expected to simulate real-world communication platforms and provide a solid foundation for further enhancements.

1.3 Scope of the project

This project focuses on the development of a real-time communication platform with core functionalities including messaging and audio/video calling. The scope of the project is defined in terms of both functional limitations and technological boundaries to ensure feasibility within the given timeframe and resources.

- **Functional limitations:** Although the system aims to simulate a modern communication platform, several limitations are defined to keep the project manageable
  - The system supports real-time one-to-one messaging and group chat but does not include advanced features such as message reactions, message editing history, or thread-based conversations.
  - Audio and video calling functionalities are limited to one-to-one communication using WebRTC, without support for group video calls or advanced conferencing features.
  - File sharing is supported for basic file types (e.g., images and documents), but does not include large file optimization, compression, or cloud storage integration.
  - The presence system (online/offline status, typing indicator) is implemented at a basic level and may not handle edge cases such as network interruptions or multi-device synchronization.
  - Security features are limited to basic authentication (JWT) and do not include advanced mechanisms such as end-to-end encryption or multi-factor authentication.
  - The system is designed for demonstration and academic purposes and therefore does not fully address all real-world production requirements.

- **Technological scope:** The project is implemented using a modern web technology stack, focusing on real-time communication and system scalability
    - The frontend is developed using a web to provide an interactive and responsive user interface.
    - The backend is implemented using Node.js with Express and Socket.io to handle HTTP requests and real-time WebSocket connections.
    - MongoDB is used as the primary database for storing message logs due to its flexibility and high performance in handling large volumes of unstructured data.
    - Redis is utilized for caching frequently accessed data and enabling Pub/Sub mechanisms to support scalability across multiple server instances.
    - Real-time communication is achieved through WebSocket, while peer-to-peer media streaming is handled using WebRTC.
    - The system is deployed using Docker and Docker Compose, with Nginx acting as a reverse proxy and load balancer to distribute traffic across backend services.
    - The architecture is designed with scalability in mind, allowing horizontal scaling and efficient handling of concurrent users.

1.4 Research methodology

To complete this project, a combination of experimental research and modern software development practices was adopted. The methodology is divided into four main phases.

First, a literature review and technology selection phase was conducted. The team studied real-time communication technologies such as WebSocket (via Socket.io) and WebRTC for peer-to-peer media transmission. Different backend

frameworks, including Node.js and Django, were analyzed to determine the most suitable solution for handling high I/O operations. MongoDB was selected for message storage due to its flexibility and scalability in managing unstructured data. In addition, Firebase Authentication was explored to support secure third-party login via Google Sign-In.

Second, the system analysis and design phase focused on defining the overall architecture and data flow. Database schemas and system diagrams were designed to represent relationships and communication processes. Special attention was given to scalability by incorporating Redis Pub/Sub to synchronize messages across multiple server instances. In addition, basic UI/UX flows were outlined to ensure a user-friendly experience.

Third, during the implementation phase, the system was developed using a modular architecture, including components such as authentication, real-time chat, and video calling. Docker was used to containerize system components (backend, database, Redis, and Nginx), while Nginx was configured as a reverse proxy to support load balancing. Docker Compose was applied to manage and deploy the entire system efficiently.

Finally, testing and evaluation were conducted to ensure system reliability and performance. Functional testing was performed to verify all core features, including messaging, group chat, and video calling. In addition, basic stress testing was carried out by simulating multiple concurrent users to evaluate system stability, response time, data consistency, and scalability, particularly in scenarios involving Redis-based synchronization.

1.5 Practical significance

The Real-Time Chat & Video Call Platform is not only a technical implementation but also provides meaningful practical value for different target groups.

For individual users and organizations, the system offers a stable and real-time communication environment that eliminates geographical barriers. In the context of

remote work and online learning, the platform enhances communication efficiency through group messaging and high-quality video calling. The integration of WebRTC enables low-latency audio and video transmission, delivering a more natural and seamless interaction experience.

For small and medium-sized enterprises, the system provides a cost-effective alternative to third-party communication platforms. Organizations can deploy an internal system based on this architecture to ensure better data privacy and control. Features such as multimedia file sharing and message history storage support knowledge management and improve collaboration, allowing managers to easily track discussions and retrieve important information.

For the software development and academic community, this project demonstrates the practical application of modern technologies such as containerization with Docker and real-time scaling using Redis. It serves as a valuable reference for understanding how to build scalable systems, distribute traffic using reverse proxy solutions like Nginx, and synchronize real-time data across multiple server instances. As a result, the project contributes to enhancing technical skills and addressing performance challenges commonly found in traditional web applications.

CHAPTER 2. LITERATURE REVIEW

2.1 Technologies Used in the KittaChat Project

In the development of the KittaChat system, several modern technologies were selected based on their suitability for real-time processing, scalability, and seamless integration within a web-based environment.

2.1.1 Backend Technologies

The backend system is developed using Node.js as the runtime environment, leveraging its non-blocking and event-driven architecture to efficiently handle concurrent connections. Express.js is used as a lightweight framework to process HTTP requests such as authentication, user management, and initial session handling.

For real-time communication, Socket.IO is utilized to enable bidirectional data exchange between clients and the server. It supports event-driven messaging and simplifies connection management, making it suitable for building real-time chat features.

To support system scalability, Redis is integrated as both a caching layer and a Pub/Sub message broker. As a Pub/Sub system, it enables synchronization of real-time events across multiple backend instances. As a caching layer, it stores frequently accessed data such as recent messages, improving system performance and reducing database load.

Additionally, the backend also acts as a signaling server for establishing peer-to-peer connections using WebRTC. It handles the exchange of signaling data such as session descriptions and ICE candidates between clients.

For deployment and scalability, Docker is used to containerize backend services and deploy multiple instances, while Nginx is configured as a reverse proxy and load balancer to distribute incoming traffic across these instances. This architecture enhances system reliability and supports horizontal scaling under high user loads.

In addition to JWT-based authentication, the system also integrates Firebase Authentication to support Google Sign-In. Firebase is used to handle third-party authentication securely, allowing users to log in using their Google accounts. After successful authentication, user information is processed and integrated into the system for further session management.

2.1.2 Frontend technologies

The frontend of the KittaChat system is developed using ReactJS, which is responsible for building a dynamic and responsive user interface. React enables efficient rendering and real-time updates of the chat interface without requiring page reloads, providing a smooth user experience.

JSX is used within React components to define the structure of the application, allowing developers to write HTML-like syntax directly in JavaScript. Tailwind CSS is applied for styling, offering a utility-first approach to create a clean, responsive, and modern user interface.

JavaScript is used to handle client-side logic and to integrate with the Socket.IO client for real-time communication. Through this integration, the frontend can send and receive messages instantly, update user status (online/offline), and display typing indicators in real time.

In addition, the frontend integrates with WebRTC APIs to support peer-to-peer audio and video calling. It manages media devices (camera and microphone), handles call interfaces (e.g., call popup, mute/unmute, video toggle), and processes signaling data received from the backend server.

ReactJS is selected due to its component-based architecture, efficient state management, and strong compatibility with real-time data-driven applications, making it suitable for building scalable and interactive communication systems.

2.1.3 Database Technology

MongoDB is used as the primary database system for storing user data and chat message history in the KittaChat platform. Its document-oriented model allows flexible storage of semi-structured data such as messages, media references, and user information, making it well-suited for real-time communication systems.

During development, MongoDB is deployed using Docker containers to provide a consistent and isolated local environment for testing and debugging. This approach ensures ease of setup and compatibility across different development machines.

In addition, MongoDB Atlas (cloud-based service) is utilized for production or remote access scenarios. It offers high availability, scalability, and managed database services, allowing the system to operate reliably without manual database maintenance.

The combination of Docker-based MongoDB for development and MongoDB Atlas for deployment provides flexibility, scalability, and ease of management for the overall system.

2.1.4 Development Tools

- **Visual Studio Code (VS Code):** Used as the primary code editor for frontend and backend development.
- **Node Package Manager (npm):** Used to manage dependencies such as ReactJS, Socket.IO, and Express.js.
- **Docker Desktop:** Docker Desktop is used to manage and run containerized services locally, including backend, database, Redis, and Nginx. It provides a consistent development environment and simplifies system deployment using Docker Compose.
- **Google Chrome:** Used for testing, debugging, and monitoring real-time WebSocket communication.

2.2 WebSocket Architecture and Workflow

2.2.1 WebSocket Architecture

The WebSocket architecture consists of several key components that work together to establish and maintain a persistent, bidirectional communication channel.

In this project, real-time communication is implemented using Socket.IO, which is built on top of the WebSocket protocol. Therefore, the overall communication model follows the same fundamental principles as WebSocket.

- **Client**

  The client is typically a web browser or mobile application that initiates the WebSocket connection. It sends the initial handshake request and communicates with the server using WebSocket frames once the connection is established.

- **Server**

  The WebSocket server receives the handshake request, upgrades the connection from HTTP to WebSocket, and manages all subsequent communication. It is responsible for handling multiple concurrent connections, broadcasting messages, and pushing updates to clients.

- **HTTP Upgrade Mechanism**

  WebSocket begins as a standard HTTP request. During the handshake, the client sends an Upgrade header requesting the transition to the WebSocket protocol. If the server accepts, the connection is upgraded, and a persistent TCP channel is established.

- **Subprotocols**

  WebSocket supports optional subprotocols that define higher-level communication rules. These subprotocols allow applications to implement structured messaging patterns.

❖ **Message Framing**

After the connection is established, data is transmitted using lightweight WebSocket frames. These frames can contain text or binary data and are significantly smaller than traditional HTTP messages, reducing overhead and improving performance.

2.2.2 WebSocket Connection Workflow

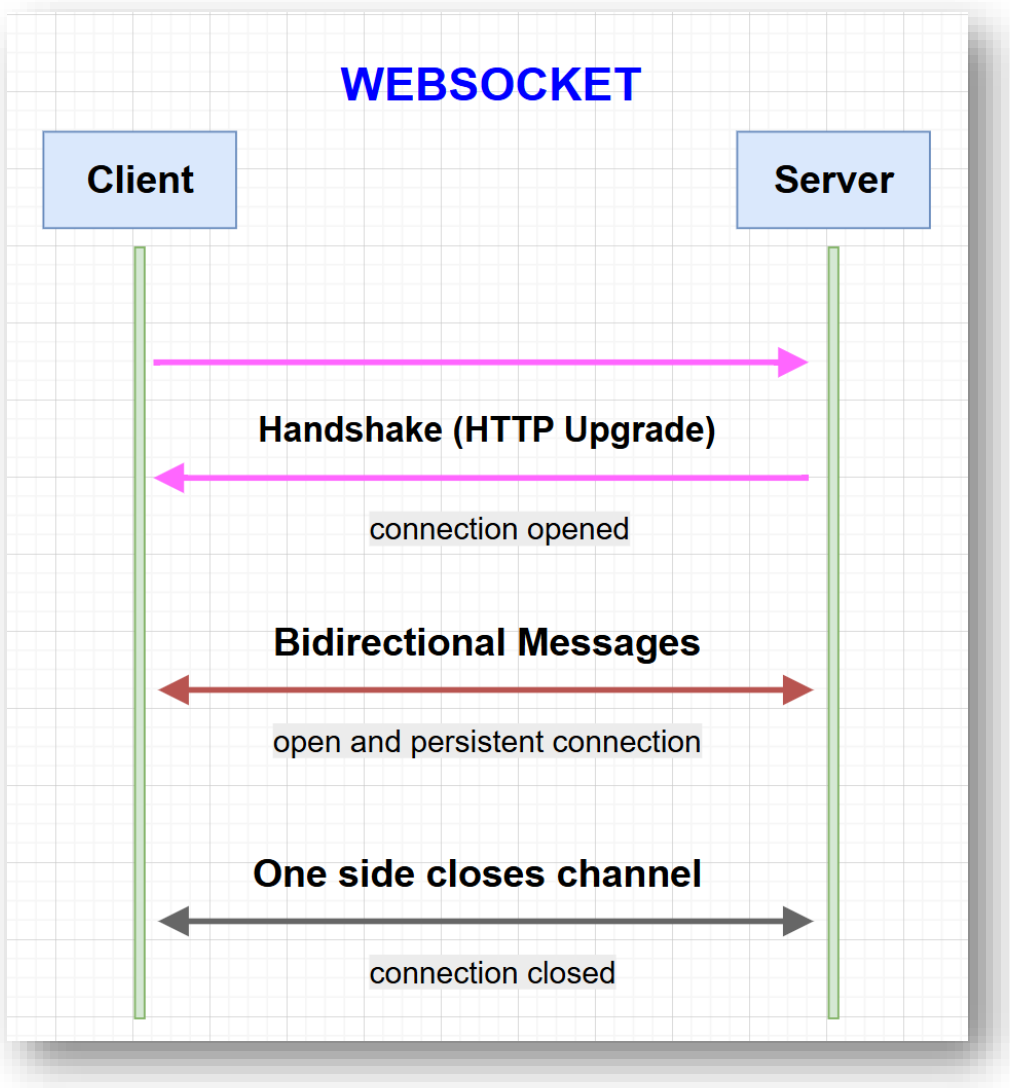

Figure 4 *WebSocket connection workflow*

The WebSocket workflow describes the sequence of steps involved in establishing, using, and closing a WebSocket connection.

- ***Step 1:*** *Client Sends HTTP Handshake Request*

  The client initiates a WebSocket connection by sending a special HTTP request to the server. This request includes the Upgrade header, indicating the intention to upgrade the communication protocol from HTTP to WebSocket.

- ***Step 2:*** *Server Accepts Protocol Upgrade*

  If the server supports WebSocket, it responds with the HTTP status code **101 Switching Protocols**, confirming that the protocol has been successfully upgraded.

- ***Step 3:*** *Establishment of Persistent WebSocket Connection*

  After the handshake is completed, the HTTP connection is replaced by a persistent WebSocket connection, allowing continuous communication between the client and the server.

- ***Step 4:*** *Real-Time Data Exchange Using Frames*

  Once the connection is established, data is transmitted between the client and server using WebSocket frames. This enables efficient, low-latency communication for real-time applications.

- ***Step 5:*** *Connection Termination*

  The connection remains active until either the client or server sends a close frame. This allows the connection to be terminated gracefully and system resources to be released properly.

2.3 WebRTC Architecture and Workflow

2.3.1 WebRTC Architecture

The WebRTC architecture enables real-time peer-to-peer communication for audio and video streaming between users. It consists of several key components that work together to establish and maintain media connections.

- **Client**

  The client is responsible for capturing audio and video streams using the user's camera and microphone through browser APIs such as MediaDevices. It also handles rendering incoming media streams from remote peers.

- **Peer Connection**

  The peer connection establishes a direct communication channel between two clients. It is responsible for transmitting audio and video streams efficiently using secure protocols.

- **Signaling Server**

  The signaling server is used to exchange connection data between clients, including session descriptions and ICE candidates. In this system, signaling is implemented using Socket.IO, enabling real-time exchange of connection information.

- **STUN/TURN Servers**

  STUN servers help clients discover their public IP addresses and network configuration, while TURN servers relay data when a direct peer-to-peer connection cannot be established due to network restrictions.

2.3.2 WebRTC Workflow

The WebRTC workflow describes the process of establishing and maintaining a peer-to-peer connection between two users for real-time communication.

1. **Call Initiation**: A user initiates a call, triggering the creation of a WebRTC peer connection and capturing local media streams.

2. **Signaling Process**: The caller sends an offer (SDP) to the callee through the signaling server. The callee responds with an answer (SDP), and both parties exchange ICE candidates to establish network connectivity.
3. **Peer-to-Peer Connection Establishment**: Once signaling is complete, a direct peer-to-peer connection is established between the two clients.
4. **Media Transmission**: Audio and video streams are transmitted directly between peers in real time, minimizing latency and reducing server load.
5. **Call Termination**: The connection is closed when either user ends the call, releasing system resources and stopping media transmission.

2.4 Redis and Real-time Scaling

Redis is an in-memory NoSQL key-value data store widely used to enhance system performance and support real-time scalability due to its low latency and efficient data access. It functions both as a caching layer and a message broker in distributed systems.

In real-time applications that require horizontal scaling, multiple backend instances are deployed to handle large numbers of concurrent users. However, this introduces challenges in maintaining consistent communication across different server instances. Redis Pub/Sub addresses this issue by enabling message broadcasting through channels, where events published by one server are immediately received by others.

In the context of WebSocket-based communication using Socket.IO, Redis can be integrated as an adapter to synchronize events across all backend instances. This ensures that users connected to different servers can still exchange messages seamlessly, maintaining consistency in real-time interactions.

Additionally, Redis is commonly used as a caching layer to store frequently accessed data such as user profiles, friend lists, and recent messages. By reducing repeated queries to the primary database (e.g., MongoDB), caching significantly improves system performance and reduces latency.

In a horizontally scaled architecture, incoming client requests are distributed across multiple backend instances using a load balancer such as Nginx. While Nginx ensures efficient traffic distribution, Redis Pub/Sub guarantees that real-time events are synchronized across all servers. This approach also supports a stateless backend architecture, where each server instance can process requests independently without maintaining shared session state.

By combining caching, distributed message synchronization, and load balancing, Redis plays a critical role in enabling high-performance, scalable, and real-time communication systems.

CHAPTER 3. REQUIREMENT ANALYSIS

3.1 User Requirements

User Group	Role Description	User Requirements
End Users (General Users)	Primary users who use the system for communication	Users need to create accounts, log in (including Google login via Firebase Authentication), and manage their profiles. They require real-time messaging with no delay, the ability to view chat history, and send multimedia files. Users also expect features such as group chat, online/offline status, typing indicators, and audio/video calling using WebRTC. The system should provide a simple, intuitive, and responsive interface.
Administrators	Manage and monitor the system operation	Administrators need the ability to monitor system performance, manage user accounts, and ensure system stability. Although not fully implemented in this project, the system should allow future extension for administrative functionalities.
System Requirements	System-level behavior	The system must support real-time communication, handle multiple concurrent users, ensure stable connections, and maintain data consistency across servers. It should provide fast response time and support scalability using technologies such as WebSocket and Redis.

Table 1 *User Requirements*

3.2 Functional Requirement

ID	Function name	Description
FR-01	User Registration	Users can register a new account using email and password.
FR-02	User Login (JWT)	Users can log in using credentials with JWT-based authentication.
FR-03	Google Login	Users can log in using Google accounts.
FR-04	User Search	Users can search for other users by email or username.
FR-05	Friend Management	Users can send, accept, or reject friend requests.
FR-06	Friend Request Notification	Users receive real-time notifications when they receive friend requests.
FR-07	Private Chat	Users can send and receive real-time messages without page reload.
FR-08	Message Storage	Chat messages are stored in the database and can be retrieved when reopening conversations.
FR-09	Message Status	The system displays message states such as “sent” and “seen”. Messages are automatically marked as “seen” when viewed.
FR-10	Group Chat	Users can create group chats and communicate with multiple members.
FR-11	Add Members to Group	All group members can add new participants to the group.
FR-12	Remove members from Group	Only the group administrator can remove members from the group.
FR-13	Delete Group (Admin)	Only the group administrator can delete or dissolve the group.

FR-14	Leave Group	Users can leave the group voluntarily. If the user is the group administrator, ownership must be transferred before leaving.
FR-15	Transfer Group Ownership	The group administrator can transfer ownership to another member before leaving the group.
FR-16	Multimedia Messaging	Users can send images and files within chat conversations.
FR-17	Emoji Messaging	Users can send emojis within messages to enhance communication.
FR-18	Image Preview	Users can preview images before sending them.
FR-19	Online Status	The system displays real-time online/offline status of users.
FR-20	Typing Indicator	The system shows “user is typing…” during conversations.
FR-21	WebRTC Signaling Mechanism	The system provides signaling to establish peer-to-peer connections between users.
FR-22	Audio/Video Call	Users can perform one-to-one audio and video calls using WebRTC.
FR-23	Call Controls	Users can mute/unmute microphone, enable/disable camera, and end calls.
FR-24	Incoming Call Notification	The system displays a popup interface when receiving an incoming call.
FR-25	WebSocket Communication	The system uses WebSocket (Socket.IO) for real-time communication.
FR-26	WebSocket Scaling	The system supports multiple backend instances for handling concurrent users.
FR-27	Redis Pub/Sub Sync	Messages are synchronized across servers using Redis Pub/Sub.

FR-28	Load Balancing	Nginx distributes incoming requests across multiple backend instances.
FR-29	Caching	Redis is used to cache frequently accessed data to improve performance.

Table 2 *Funtional Requirements*

3.3 Non-functional requirement

3.3.1 Performance

KittaChat is designed to provide real-time message delivery with minimal latency. Messages and events, including typing indicators, online status, and read receipts, are transmitted instantly without noticeable delay. The use of WebSocket and WebRTC ensures low-latency communication for both messaging and audio/video calls.

3.3.2 Reliability and Stability

The system must maintain stable connections and ensure consistent data delivery, even under high user load. Mechanisms such as automatic reconnection provided by Socket.IO and message synchronization via Redis improve system reliability. These features help prevent data loss and ensure continuous communication between users.

3.3.3 Security

Basic security mechanisms are implemented, including user authentication using JWT and secure communication between client and server. However, advanced security features such as end-to-end encryption are beyond the scope of this project.

3.3.4 Scalability

The system is designed to support a large number of concurrent users through horizontal scaling. Multiple backend instances can be deployed simultaneously, while

Redis Pub/Sub is used to synchronize real-time data across servers. This ensures seamless communication between users connected to different server instances in a distributed environment.

3.3.5 Availability

The system aims to provide high availability by ensuring that services remain accessible even when one or more components fail. Load balancing using Nginx and containerized deployment with Docker help maintain system uptime and fault tolerance.

3.3.6 Usability

The user interface of KittaChat is designed to be simple, intuitive, and responsive. Users can easily perform actions such as sending messages, participating in group chats, and initiating audio/video calls without complex interactions.

3.3.7 Maintainability

The system follows modular architecture, separating components such as authentication, messaging, and video calling. This improves code readability, simplifies debugging, and facilitates future development and scalability.

3.4 Use-case Diagram for KittaChat Real-time chat & video call platform

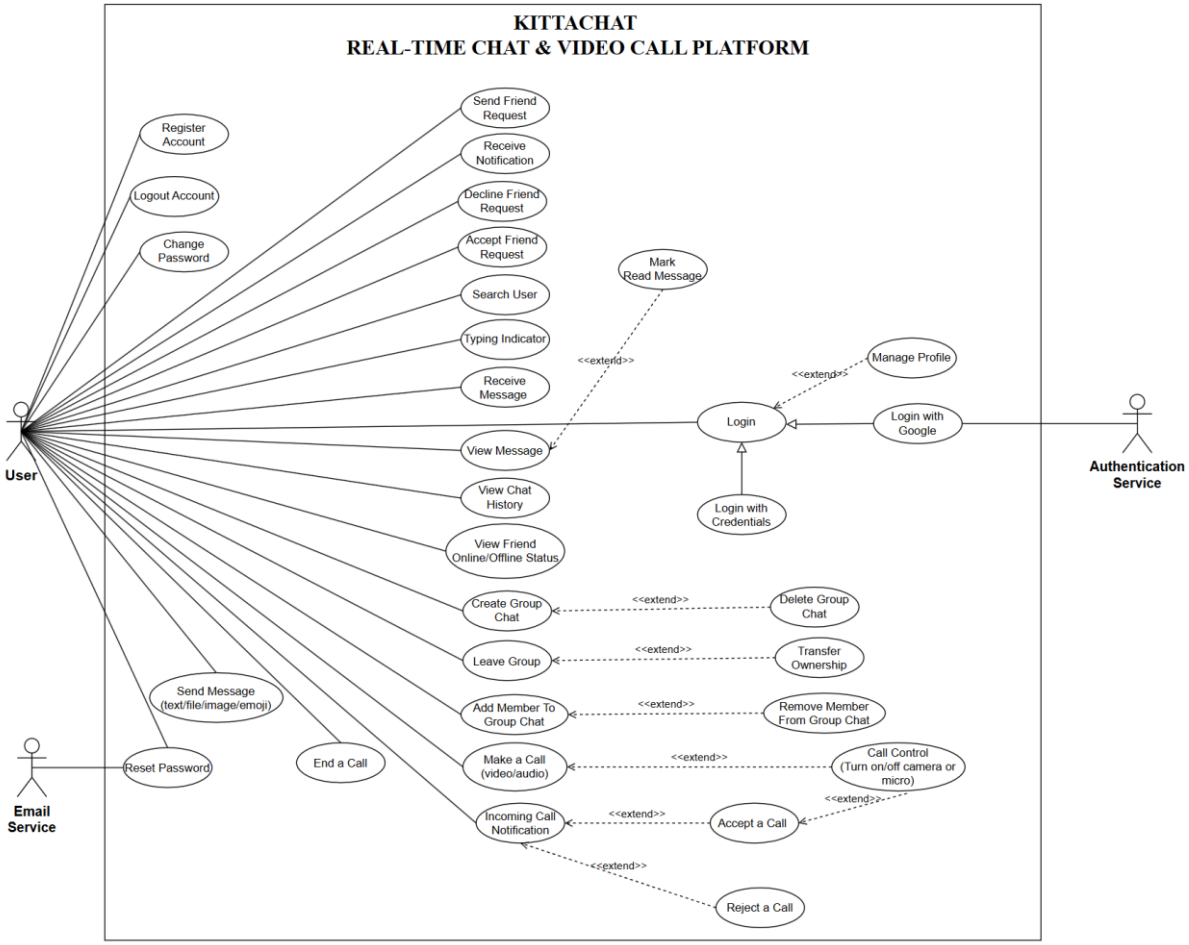

Figure 5 *Use-case Diagram for the system*

CHAPTER 4. SYSTEM DESIGN

4.1 Overview System Architecture

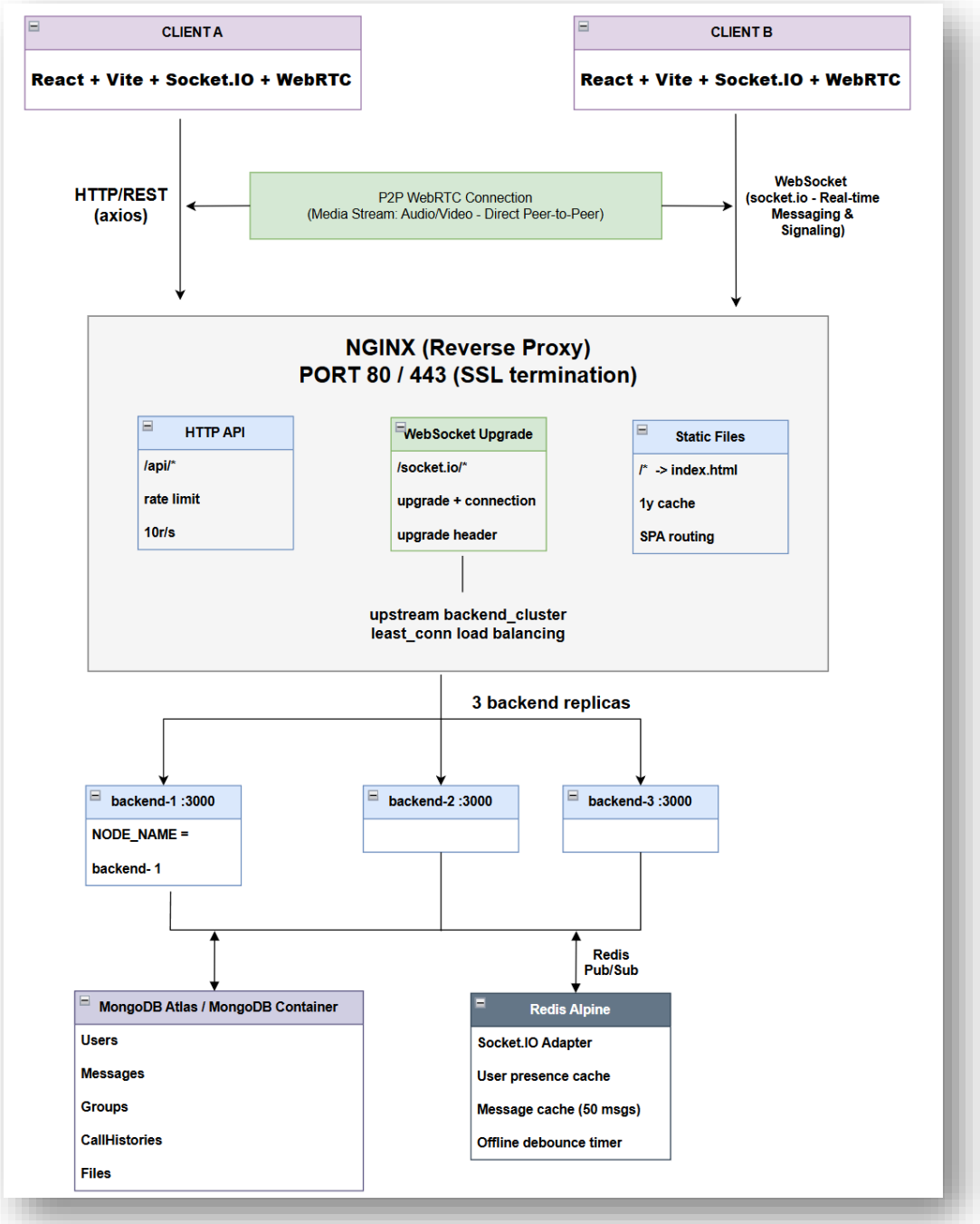

Figure 6 *Overall System Architecture*

The overall architecture of the KittaChat system is designed as a scalable real-time communication platform supporting both messaging and peer-to-peer video calling.

The system follows a client-server architecture combined with a distributed backend design to ensure scalability and high performance. Two client applications (Client A and Client B) interact with the system through both HTTP and WebSocket protocols. HTTP (via Axios) is used for REST API communication such as authentication and data retrieval, while WebSocket (Socket.IO) enables real-time messaging and signaling for WebRTC connections.

Nginx acts as a reverse proxy and load balancer, handling incoming traffic on ports 80 and 443. It routes HTTP requests to API endpoints and upgrades WebSocket connections for real-time communication. Additionally, Nginx distributes requests across multiple backend instances using a load balancing strategy (e.g., least connections), ensuring efficient handling of concurrent users.

The backend layer consists of multiple replicated Node.js server instances. These instances are designed to be stateless, enabling horizontal scaling without shared session dependencies. Real-time communication is implemented using Socket.IO. When a message is processed by one backend instance, it is propagated to other instances via Redis Pub/Sub, ensuring that users connected to different servers receive messages consistently.

For audio and video communication, WebRTC is used to establish direct peer-to-peer connections between clients. The backend only handles signaling through WebSocket, while media streams are transmitted directly between users, reducing server load and latency.

MongoDB is used as the primary database to store persistent data, including users, messages, groups, call histories, and files. Redis is additionally used as both a caching layer and a Pub/Sub message broker to enhance performance and enable real-time event synchronization across distributed backend instances.

Overall, this architecture ensures efficient real-time communication, supports high concurrency, and provides scalability through load balancing and distributed message synchronization.

4.2 Real-time Communication Workflows

4.2.1 Socket Message Flow (Socket.io)

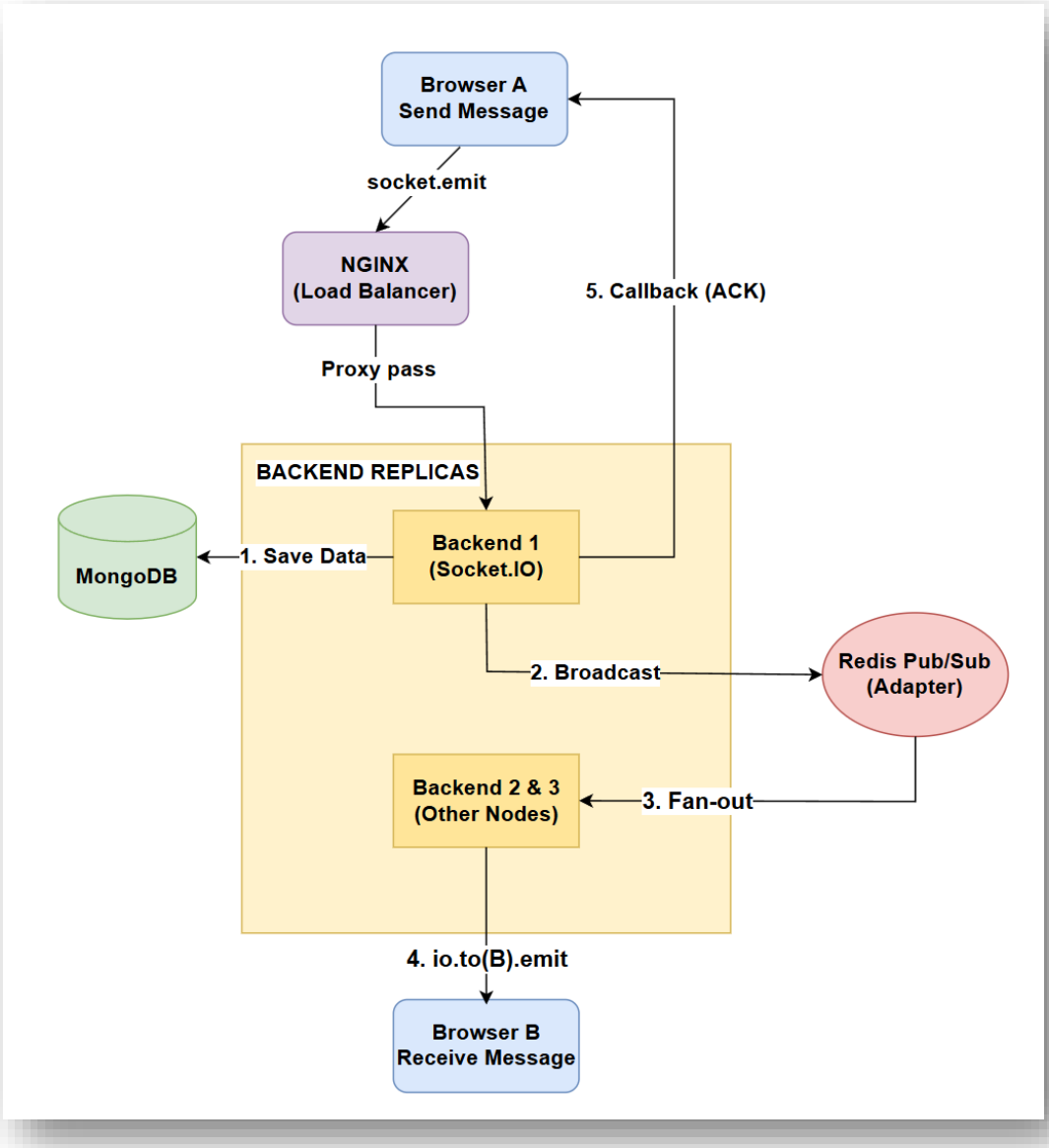

Figure 7 *Real-time message flow using Socket.IO and Redis Pub/Sub*

The real-time message flow in the system is designed to ensure reliable delivery and scalability across multiple backend instances.

When a user sends a message from the client, it is emitted via WebSocket and routed through Nginx, which acts as a reverse proxy and load balancer. The request is then forwarded to one of the backend instances.

Upon receiving the message, the backend server first persists the message in MongoDB to ensure data durability. It then publishes the message event to Redis using the Pub/Sub mechanism.

Redis propagates the event to all backend instances in the cluster, enabling cross-node synchronization. Each backend instance then processes the event and delivers the message to the appropriate connected clients using Socket.IO.

Finally, the receiving client obtains the message in real time, and an acknowledgment (ACK) is sent back to the sender to confirm successful delivery. This mechanism ensures message consistency and delivery reliability.

Overall, this architecture enables seamless communication between users connected to different backend instances while supporting horizontal scalability and maintaining consistency in real-time messaging.

4.2.1.1 Individual Chat Flow

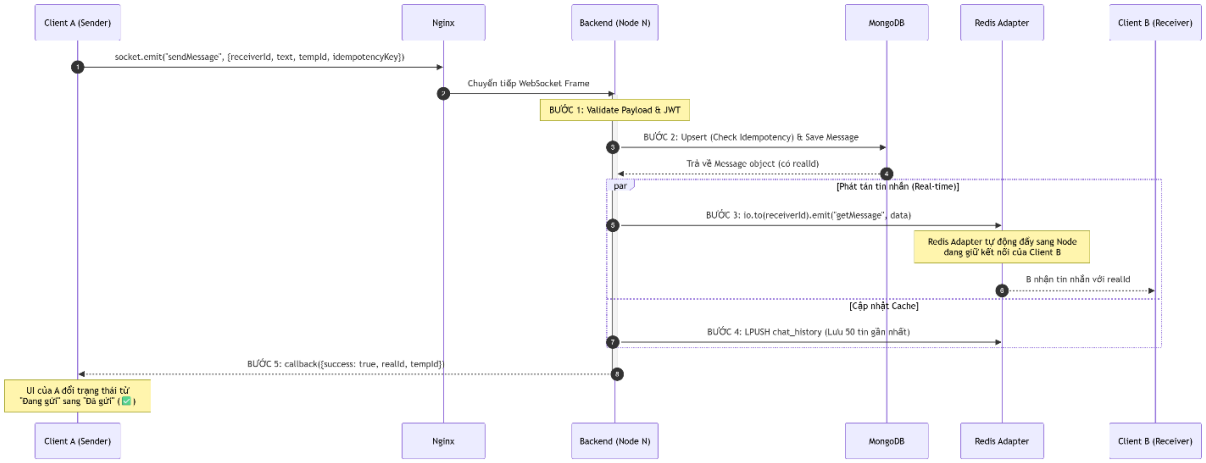

Figure 8 *The detailed sequence of message processing for one-to-one communication*

4.2.1.2 Group Chat Flow

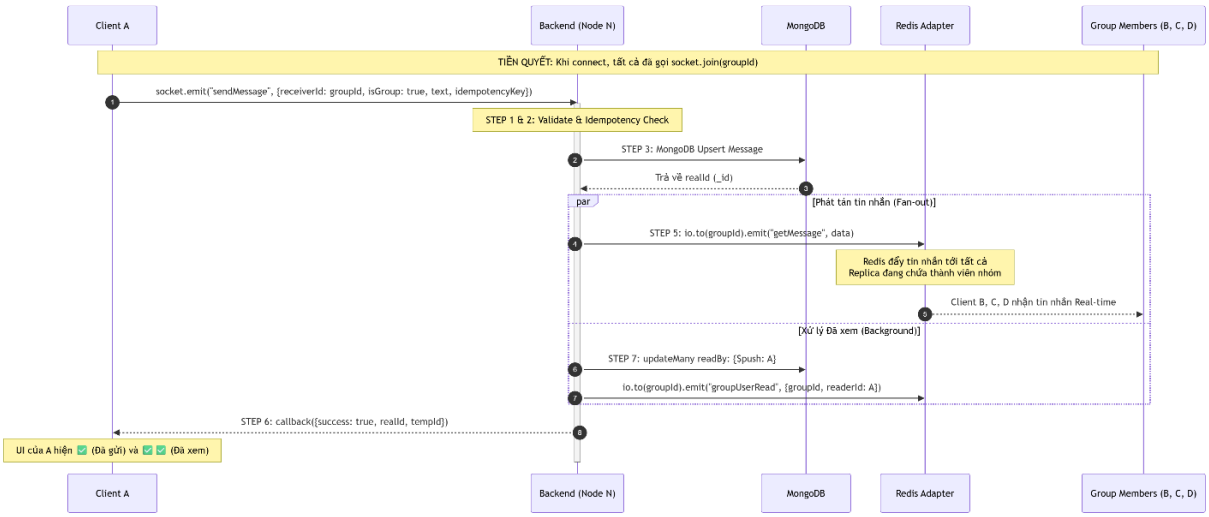

Figure 9 *The sequence flow for group messaging in the system*

4.2.2 Connection Initialization & Presence

The sequence diagram illustrates the connection initialization and presence management process in the system.

When a client establishes a WebSocket connection, it sends a JWT token for authentication. The backend verifies the token and associates the socket connection with the corresponding user ID.

After successful authentication, the client registers itself with the server, and the backend assigns the socket to a dedicated room identified by the user ID, enabling efficient one-to-one communication.

The system then retrieves all group conversations that the user participates in and automatically subscribes the socket to the corresponding group rooms. This allows the client to receive real-time updates for both individual and group messages.

For presence management, Redis is used as a centralized store to maintain user connection states across distributed backend instances. When a user connects, the backend updates Redis by removing any existing offline timers, registering the current socket in the user's active socket set, and marking the user as online.

This mechanism ensures accurate and consistent tracking of user presence in a horizontally scalable environment. It also enables real-time features such as online/offline indicators and typing status synchronization.

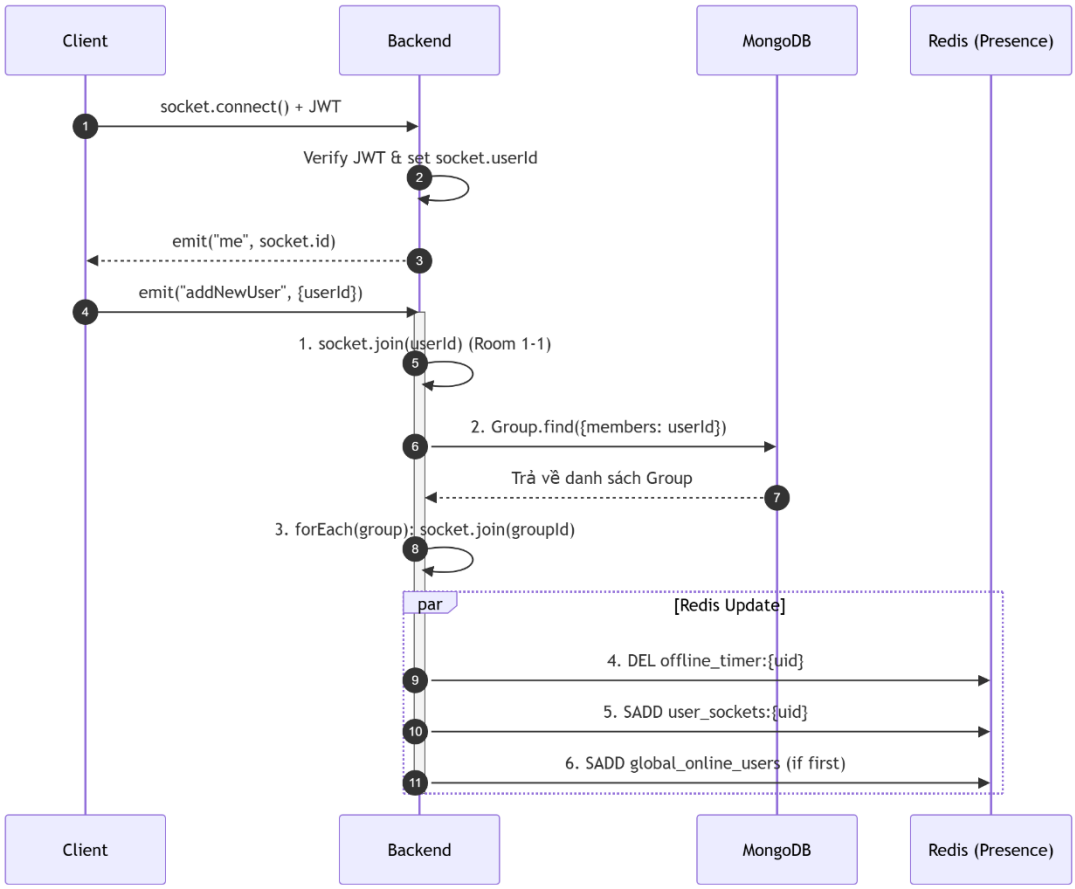

Figure 10 *Connection Initialization and Presence Management*

4.2.3 Offline Handling & Message Synchronization

The sequence diagram illustrates the message synchronization and read-receipt handling process when a user reconnects after being offline.

When Client A sends a message while Client B is offline, the backend stores the message in MongoDB with an unread status (isRead = false). Since Client B is not connected via WebSocket, the message cannot be delivered in real time.

When Client B reconnects and opens the application, it sends a synchronization request to the backend to retrieve missed messages. The backend

queries MongoDB for unread messages and returns the message list along with relevant metadata such as unread counts.

After receiving the messages, Client B marks them as read through either a WebSocket event or an API call. The backend updates the message status in the database and emits a read-receipt event to Client A, allowing the sender's UI to reflect the updated message status.

This mechanism ensures reliable message delivery and consistency, even when users experience temporary disconnections, and enhances user experience through accurate read-receipt tracking.

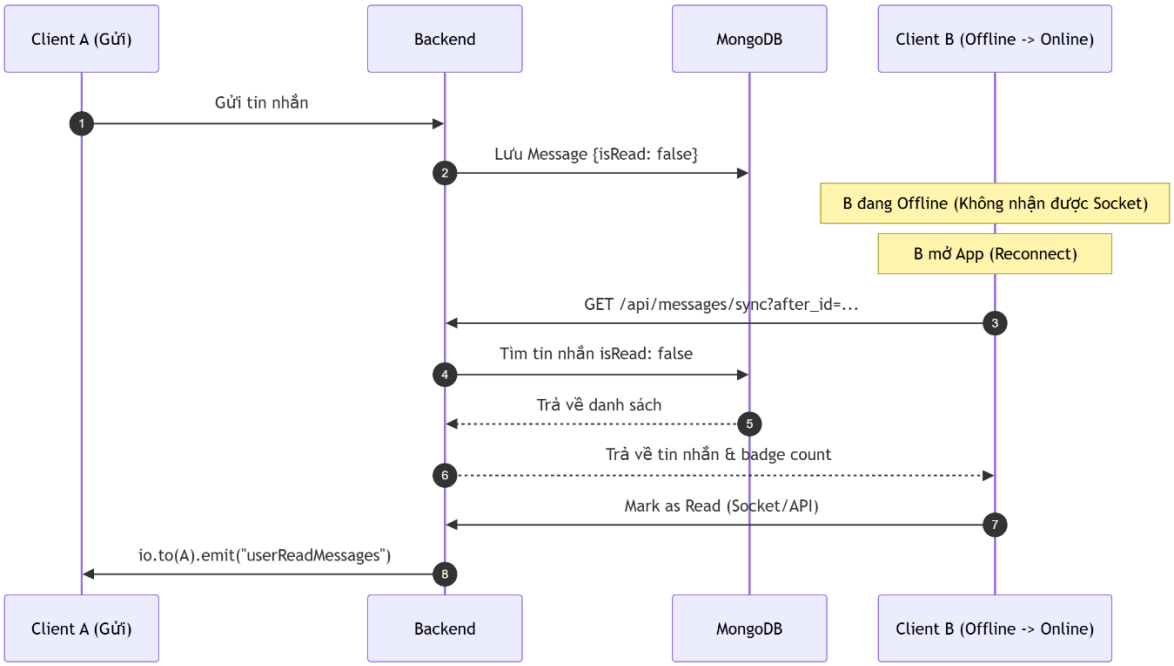

Figure 11 *Offline Handling & Message Synchronization*

4.3 WebRTC Communication Workflow

4.3.1 WebRTC Architecture

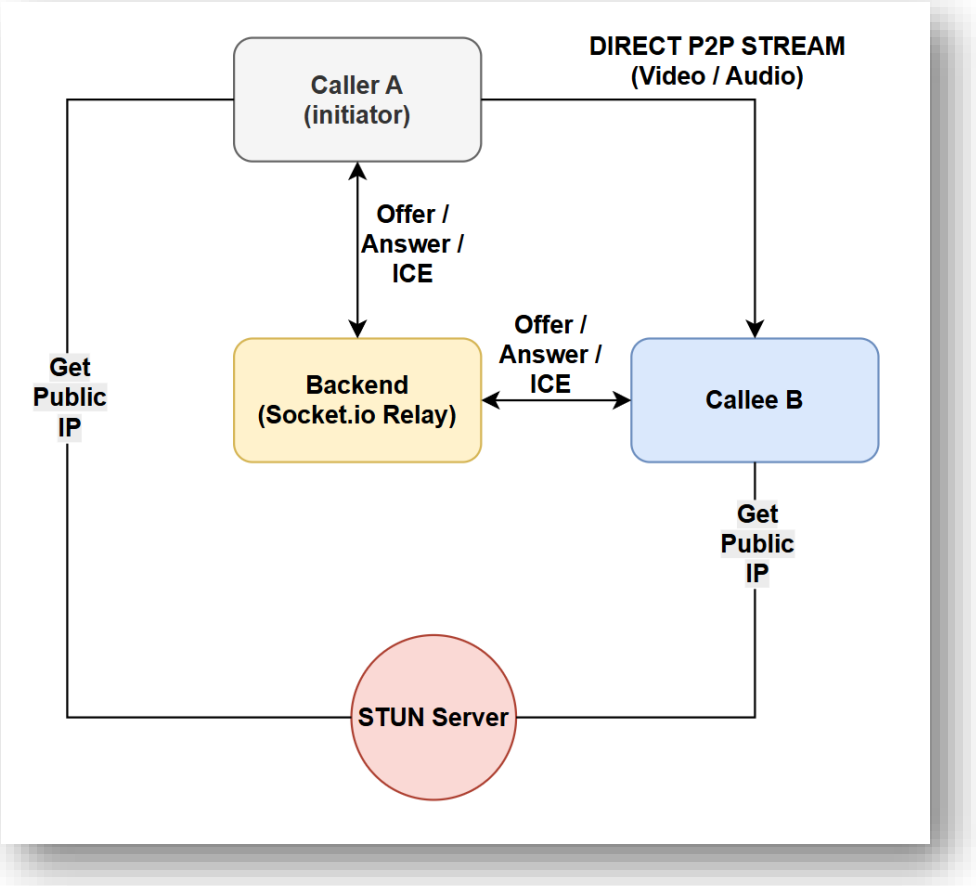

Figure 12 *WebRTC Architecture*

The WebRTC architecture in the system is designed to enable direct peer-to-peer communication between clients while using the backend server only for signaling.

When a user initiates a call, the caller (Client A) establishes a connection with the backend server via Socket.IO, which functions as a signaling relay. Through this channel, signaling data such as SDP offers, answers, and ICE candidates are exchanged between the caller and the callee.

To support connectivity across different network environments, both clients interact with a STUN server to obtain their public IP addresses and network

configuration. This process facilitates NAT traversal and helps establish a viable communication path between peers.

After the signaling phase is completed and the necessary network information is exchanged, a direct peer-to-peer connection is established between the caller and the callee. Audio and video streams are then transmitted directly between the two clients without passing through the backend server.

By separating signaling and media transmission, this architecture reduces server load and minimizes latency, making it highly efficient for real-time audio and video communication.

4.3.2 WebRTC Connection Flow

The WebRTC connection process begins when the caller initiates a call and creates an RTCPeerConnection instance. The caller then generates an SDP offer and sends it to the callee through the signaling server using WebSocket (Socket.IO).

Upon receiving the offer, the callee creates its own RTCPeerConnection instance and responds with an SDP answer, which is sent back through the signaling server. This exchange allows both peers to negotiate the connection parameters required for communication.

After the offer–answer exchange, both clients continuously share ICE candidates to determine the most suitable network path. This step enables successful connection establishment even in complex network environments such as NAT or firewall-restricted networks.

Once the negotiation process is completed, a direct peer-to-peer connection is established between the two clients. At this stage, audio and video streams are transmitted directly between peers without passing through the backend server.

The backend server is only responsible for signaling and coordination, while the actual media transmission occurs directly between clients, ensuring low latency and efficient real-time communication.

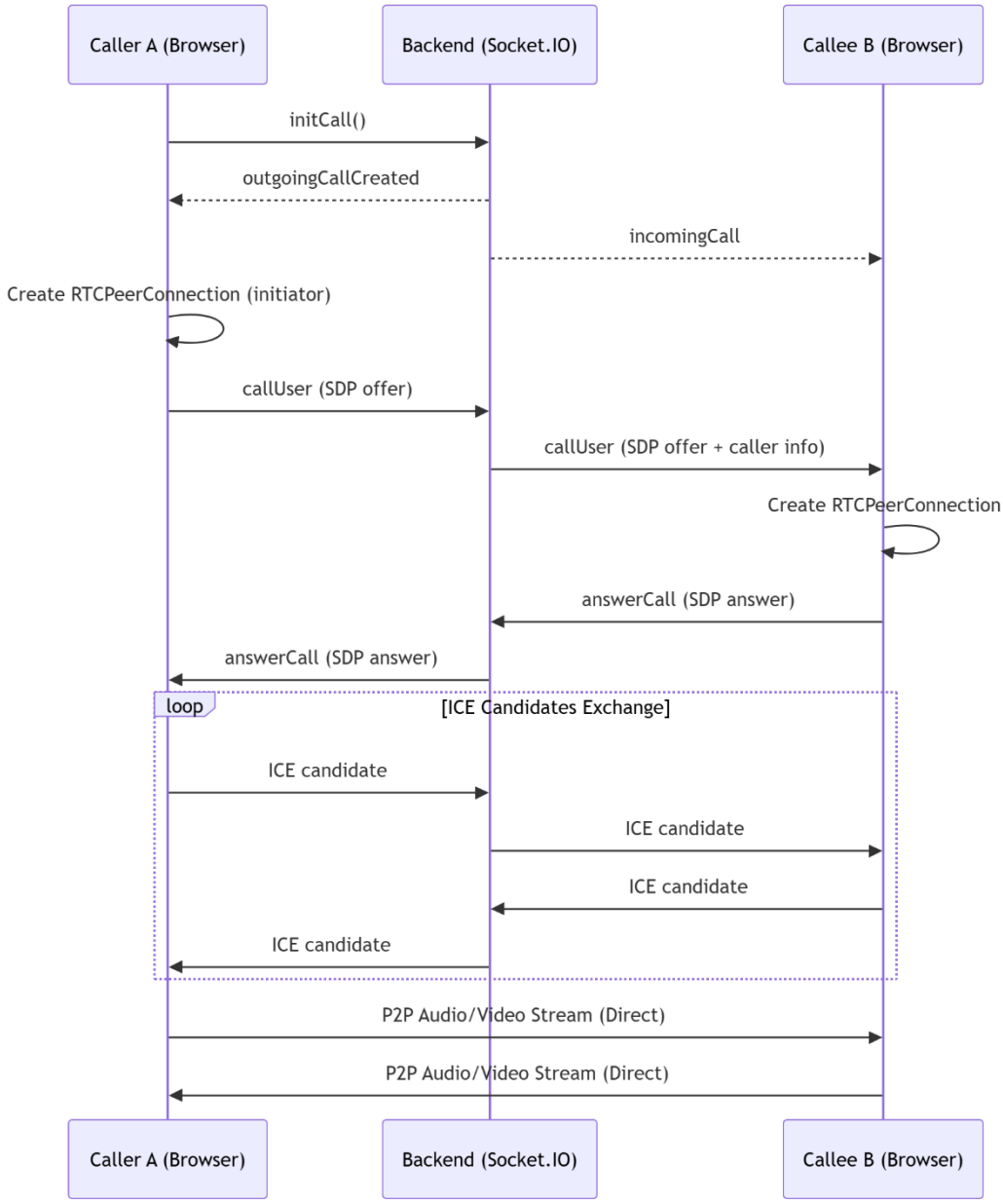

Figure 13 *WebRTC Signaling and Peer-to-Peer Connection Establishment Flow*

4.3.3 WebRTC Call State

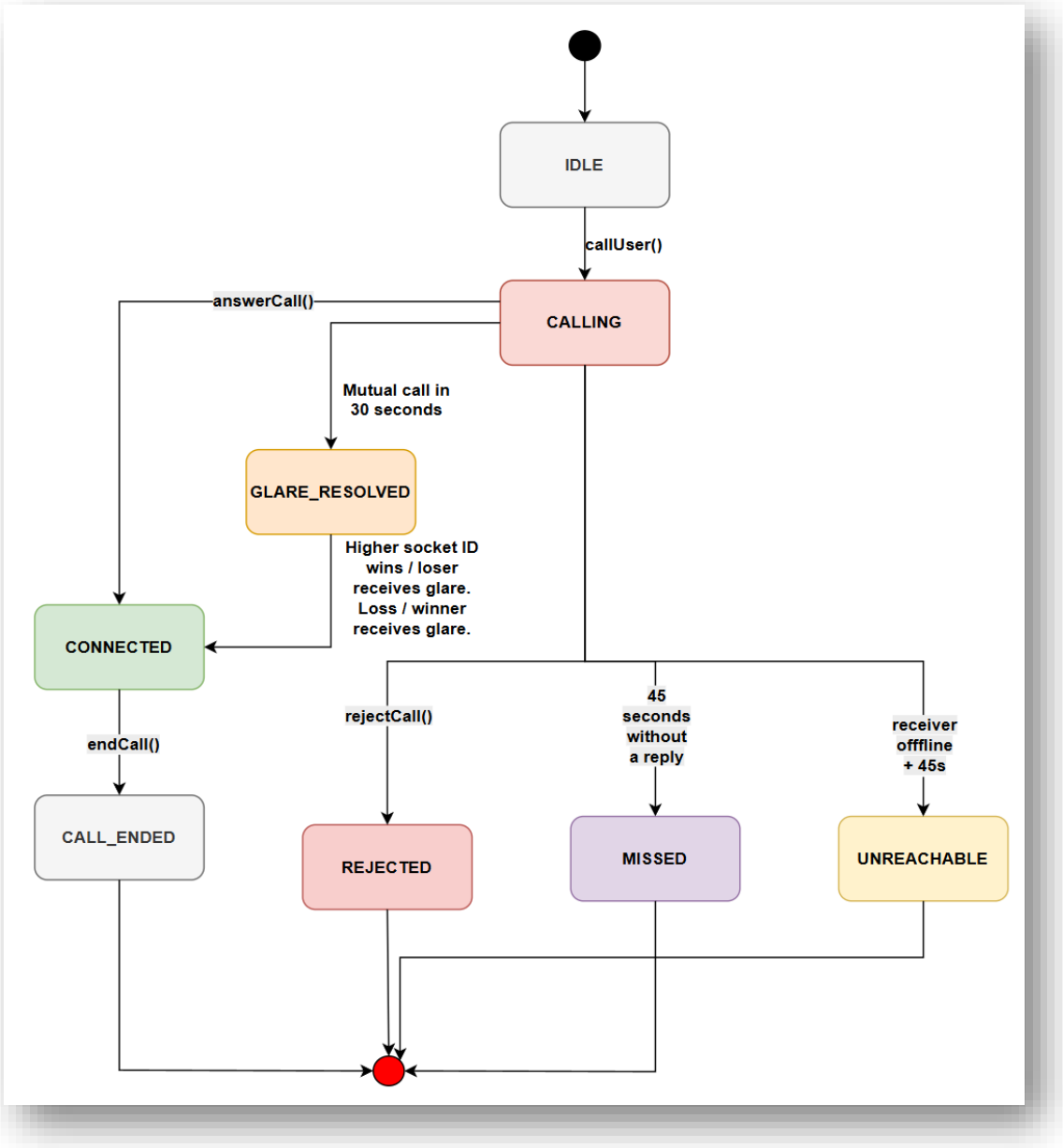

Figure 14 *WebRTC Call State Workflow and Handling Scenarios*

The WebRTC call process is managed using a state-driven workflow to ensure consistent handling of different call scenarios.

The system starts in the *IDLE* state. When a user initiates a call, the state transitions to *CALLING*. If the receiver accepts the call, the system moves to the

CONNECTED state, where a peer-to-peer WebRTC connection is established and media streams are exchanged in real time.

Several edge cases are handled to improve reliability. If the receiver rejects the call, the state transitions to *REJECTED*. If no response is received within a specified timeout, the call is marked as *MISSED*. In situations where the receiver is offline or unreachable, the system transitions to the *UNREACHABLE* state.

The system also handles simultaneous call attempts (glare scenarios). When both users initiate calls at the same time, a deterministic rule is applied to resolve the conflict and establish a single stable connection between peers.

When the call ends, the state transitions to *CALL_ENDED*, completing the call lifecycle and releasing related resources.

This state-based approach ensures robust call management, simplifies client-side logic, and provides a consistent user experience across different real-time communication scenarios.

4.3.4 Glare Handling

In real-time communication systems, a glare scenario occurs when both users initiate a call simultaneously, leading to conflicting WebRTC connection attempts if not properly handled.

To address this issue, the system implements a deterministic glare-handling mechanism. When two call requests are detected within a short time window, the backend applies a predefined rule (e.g., comparing socket IDs) to determine which client will act as the initiator.

The lower-priority client cancels its current peer connection and switches to a non-initiator role, preparing to accept an incoming offer. Meanwhile, the selected initiator continues the signaling process by generating and sending the SDP offer.

The other client receives the offer and responds with an SDP answer, after which the normal WebRTC signaling flow proceeds with ICE candidate exchange.

This approach ensures that only one valid connection is established, preventing duplicated or conflicting peer connections. As a result, a stable peer-to-

peer communication channel is successfully formed even under simultaneous call conditions.

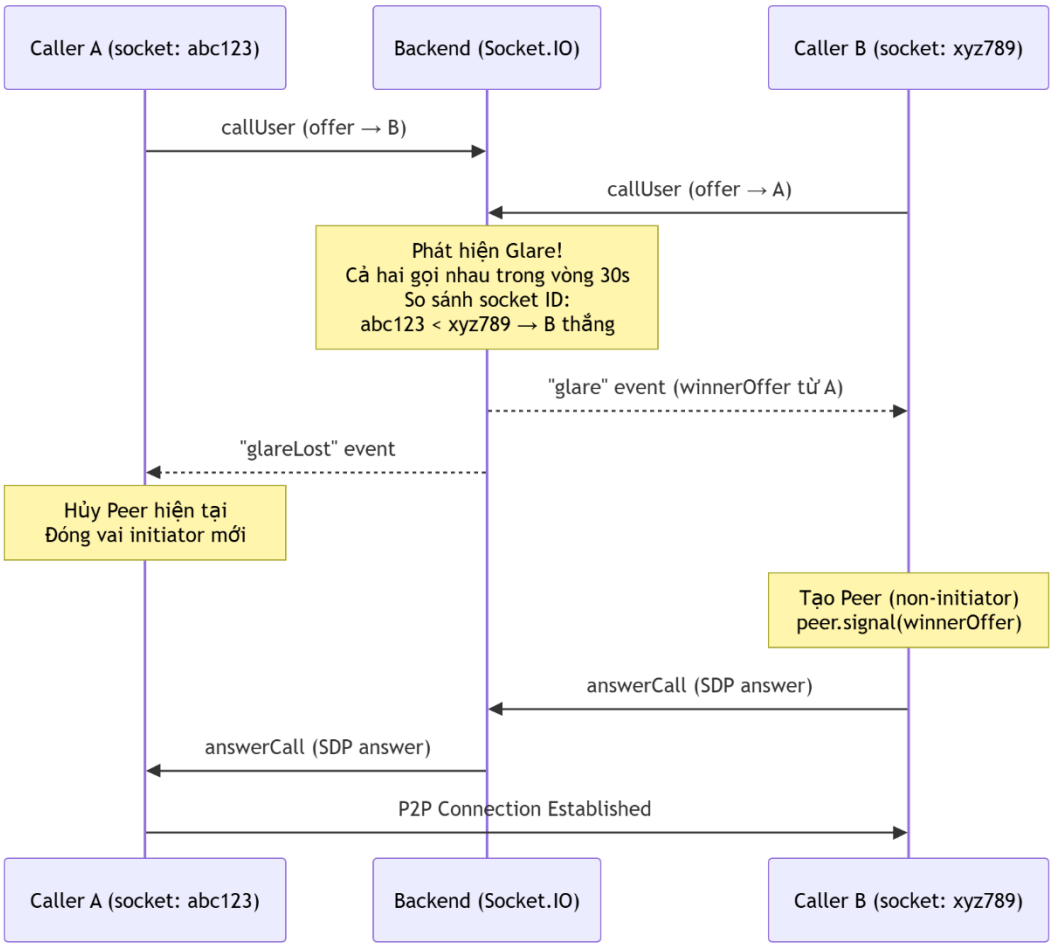

Figure 15 *WebRTC Glare Detection and Resolution Mechanism*

4.4 Database Design

4.4.1 Database Overview

The system uses MongoDB as the primary database to store application data, including users, messages, groups, call histories, and files.

A NoSQL document-based approach is chosen to efficiently handle large volumes of real-time data and support flexible data structures.

The database is designed to ensure scalability, fast read/write operations, and efficient querying for chat history and real-time communication features.

Core collections in the system include users, messages, groups, files, and call_histories, along with supporting collections for relationships such as message read status, group membership, and file attachments.

In addition, the database design follows a normalized and modular structure, where relationships between entities are managed through references, enabling flexibility and efficient data access in a distributed environment.

The detailed structure and relationships between these collections are illustrated in the following schema diagram.

4.4.2 Schema Design

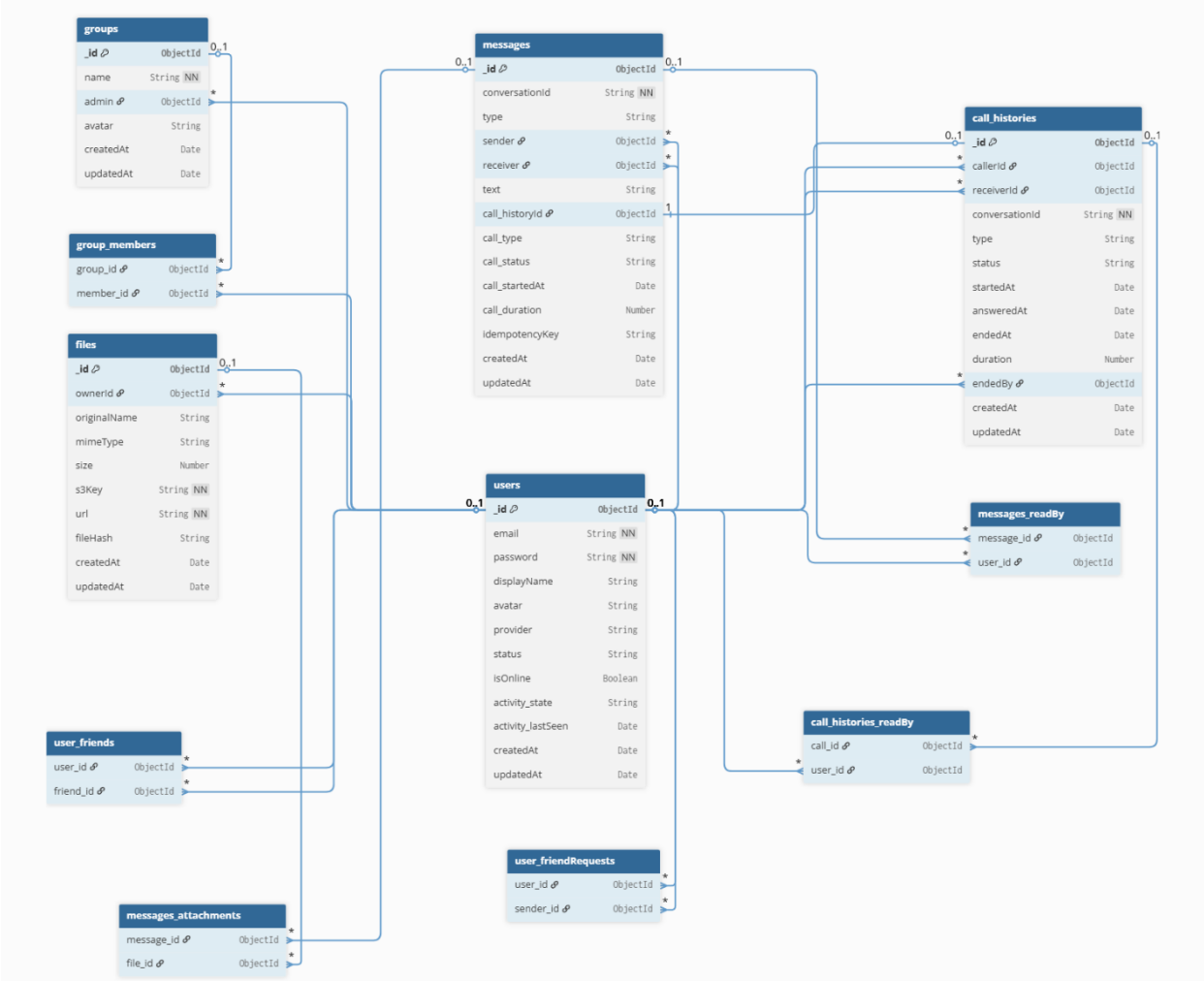

Figure 16 *Database Schema Diagram*

4.4.3 Key Entities and Schema

The database schema is designed using a modular and reference-based approach to support scalability and efficient data management in a real-time communication system. Instead of embedding large or frequently changing data, the system primarily uses references between collections to ensure flexibility, efficient updates, and data consistency.

The core entities of the system include **users**, **messages**, **groups**, **call_histories**, and **files**. In addition, several supporting structures are used to model relationships such as friendships, group memberships, message read status, and

attachments. These relationships are implemented through separate linking collections, which improves scalability and avoids document size limitations in MongoDB.

The main entities are defined as follows:

- **users**: Stores user account information, authentication data, and presence status.
- **messages**: Acts as the central message log, supporting both one-to-one and group conversations.
- **groups**: Represents group chat information, including members and administrators.
- **files**: Manages metadata of uploaded files and media attachments.
- **call_histories**: Stores records of audio and video calls, including status and duration.

Among these, the messages collection plays a central role in the system. Each message is associated with a **conversationId**, which uniquely identifies a conversation. For one-to-one chats, the **conversationId** is generated deterministically by combining and sorting user identifiers. For group chats, it corresponds directly to the group ID. This unified design simplifies data management and enables efficient querying for both chat types.

Additional relationships are implemented using references. For example, group membership is managed through a separate structure, message attachments are linked to files, and read status is tracked using a dedicated mechanism. This approach ensures flexibility and supports scalable real-time communication.

4.4.4 Key Relationships Between Entities

The database schema defines several key relationships between entities to support real-time communication and ensure data consistency across the system.

4.4.4.1 User with User (Friendship)

The system models friendships using a self-referencing relationship between users. Each user maintains a list of friends and pending friend requests, enabling efficient retrieval of social connections and real-time presence updates.

4.4.4.2 Group with Users (Membership)

Group membership is represented using a many-to-many relationship between groups and users. Each group contains a list of members, allowing users to participate in group conversations and receive real-time updates.

4.4.4.3 Message with User (Sender/Receiver)

Each message is associated with a sender and, in one-to-one conversations, a receiver. For group conversations, the receiver field is omitted, and the group context is determined using the conversationId.

4.4.4.4 Message with Group

Messages belonging to group conversations are linked to a group through the conversationId, which corresponds to the group ID. This allows all messages in a group to be retrieved efficiently.

4.4.4.5 Message with File (Attachments)

Messages can include multiple file attachments, which are stored in a separate collection. This relationship enables efficient handling of multimedia content without increasing the size of message documents.

4.4.4.6 Message with Read Status

Read status is implemented using a flexible approach. For one-to-one conversations, a simple read mechanism is used, while for group chats, a readBy structure tracks which users have read each message.

4.4.4.7 Message with Call History

Call-related messages are linked to call history records, allowing call logs to be integrated directly into the chat interface and displayed alongside regular messages.

4.4.4.8 Call History with User

Each call history record is associated with both the caller and the receiver, enabling tracking of incoming and outgoing calls, as well as call status and duration.

4.4.5 Message Log Storage Design

The messages collection is designed as a centralized message log to store all communication data, including one-to-one messages, group messages, file attachments, and call-related events.

Each message is associated with a conversationId, which uniquely identifies a conversation. For one-to-one communication, the conversationId is generated by sorting and concatenating the two user IDs. For group conversations, it directly corresponds to the group ID. This unified design allows both communication types to be handled within a single collection, simplifying data management and query operations.

The schema supports multiple message types such as text, file, system, and call_log. File attachments are stored as references to the files collection, while call-related data is embedded within the message document to enable inline rendering of call logs in the chat interface.

To ensure data consistency, the system uses an idempotencyKey mechanism. Each message request includes a unique identifier, allowing the backend to perform an upsert operation and prevent duplicate messages caused by network retries.

For read status tracking, a hybrid approach is applied. A boolean flag is used for one-to-one conversations, while a readBy array is used for group messages to track which users have read each message.

To improve performance, recent messages are cached using Redis, allowing fast retrieval of the latest messages without querying MongoDB. This reduces database load for frequently accessed conversations.

Combined with indexing strategies on conversationId, sender, and timestamps, this design enables efficient pagination, fast querying, and scalable real-time message storage.

4.5 User Interface Design

4.5.1 Login Account Interface

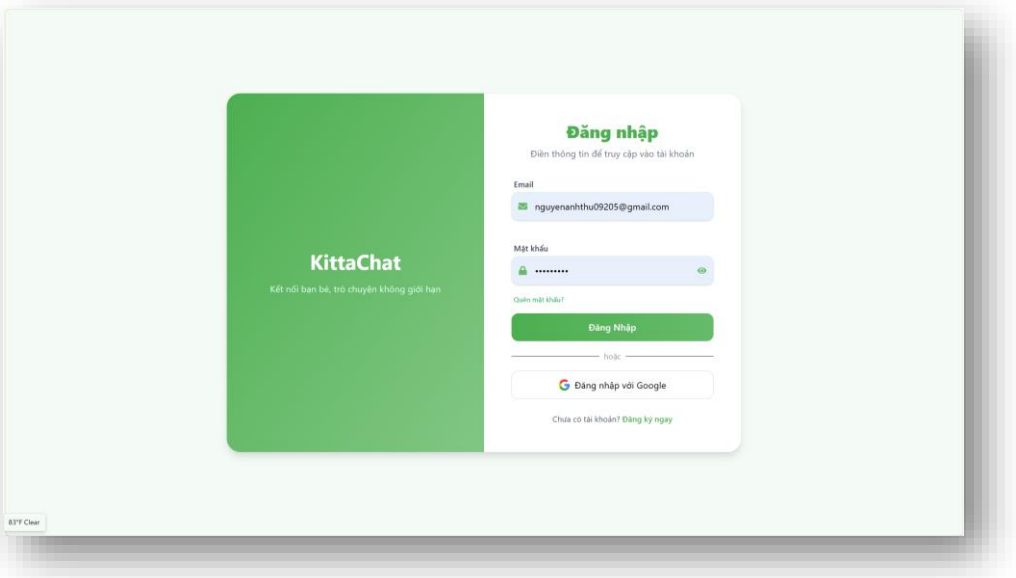

Figure 17 *UI Login Account*

4.5.2 Register Account Interface

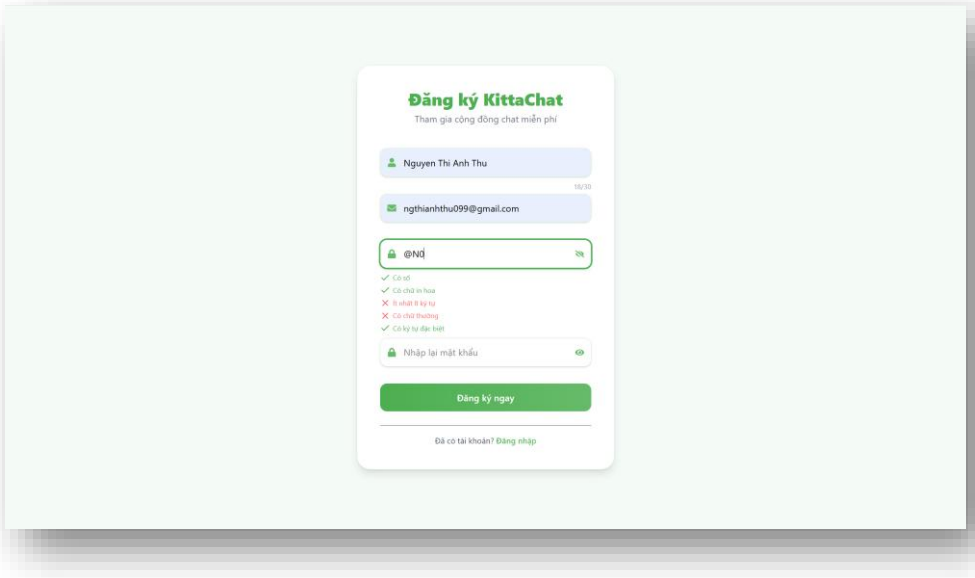

Figure 18 *UI Register Account*

4.5.3 General Interface

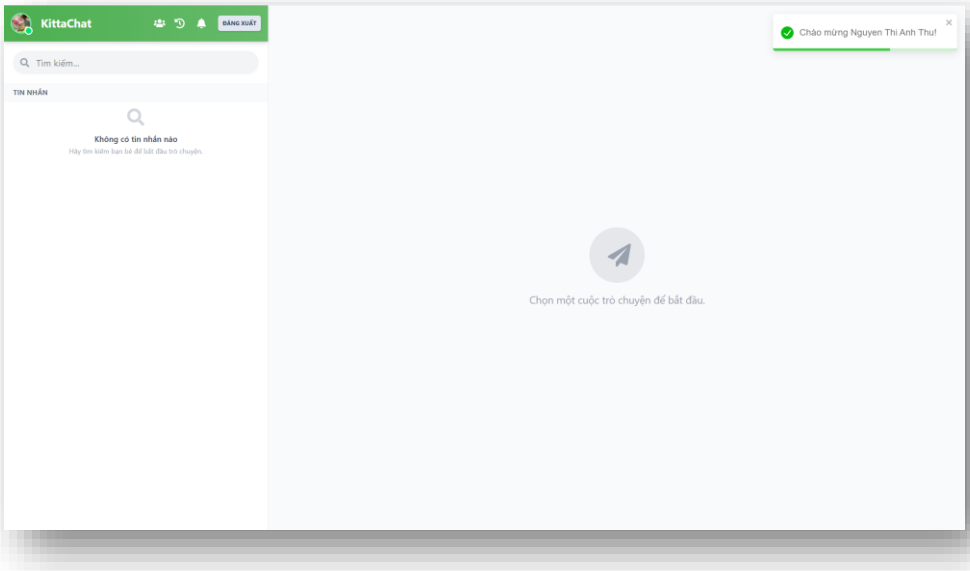

Figure 19 *UI General when there is no conversation*

4.5.4 Search for User Interface

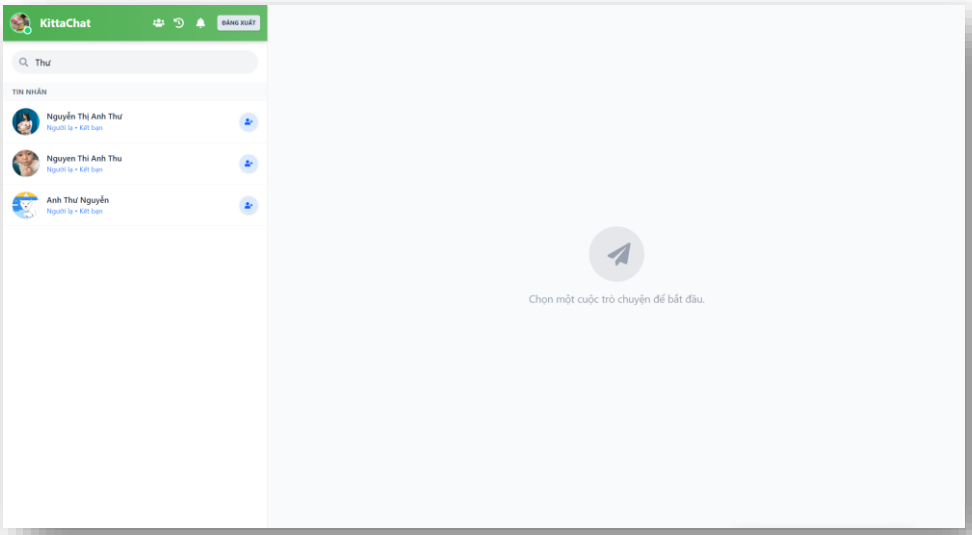

Figure 20 *UI Search User*

4.5.5 Friend Request List Interface

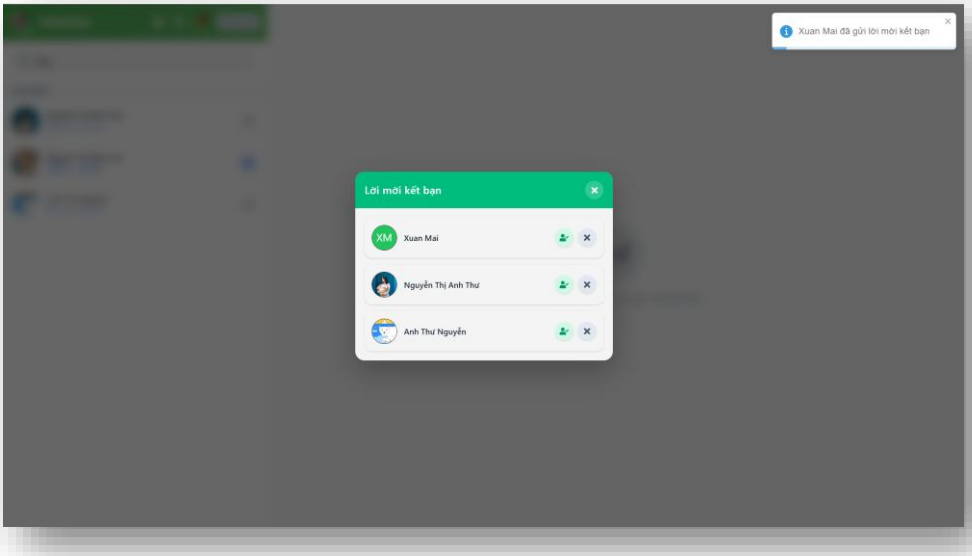

Figure 21 *UI List of Friend Request*

4.5.6 Chat One-One Interface

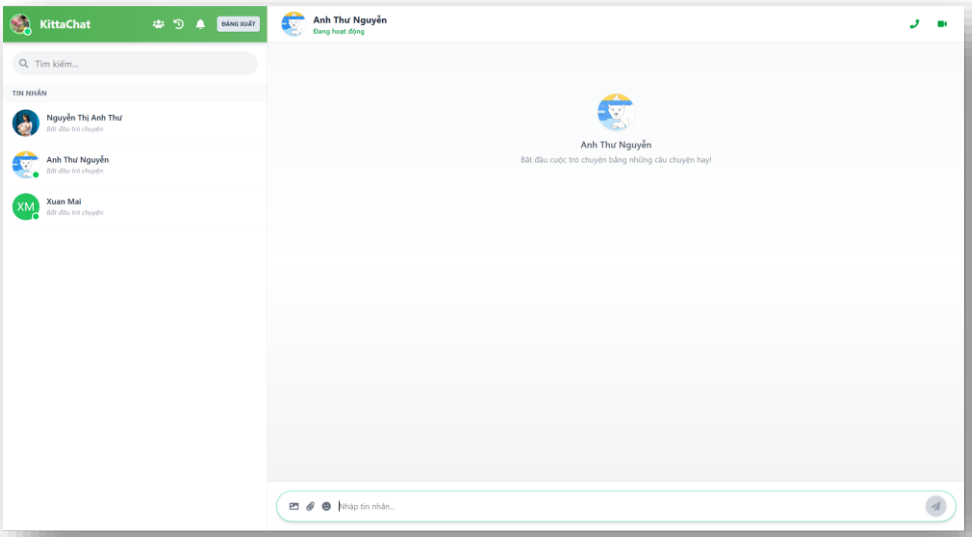

Figure 22 *UI Chat when no message yet*

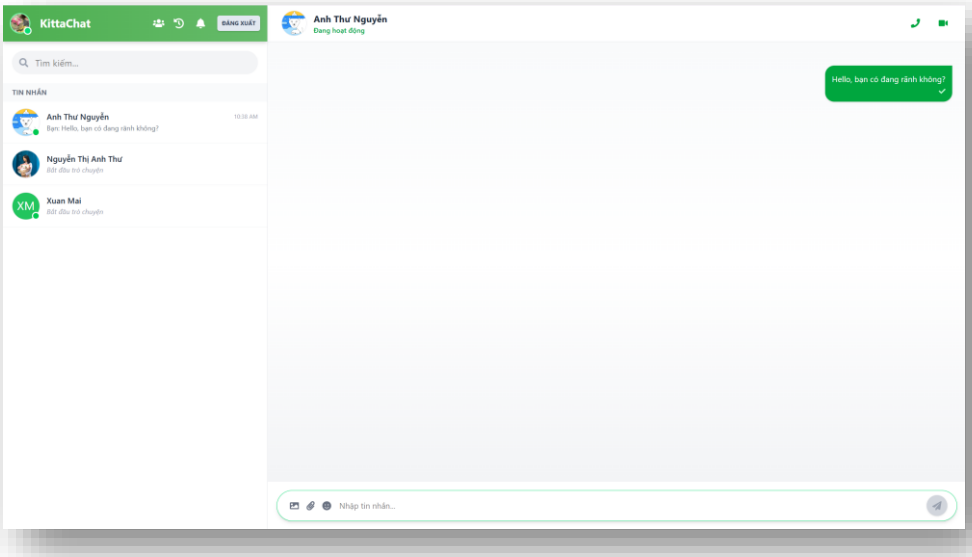

Figure 23 *UI Chat after sending a message*

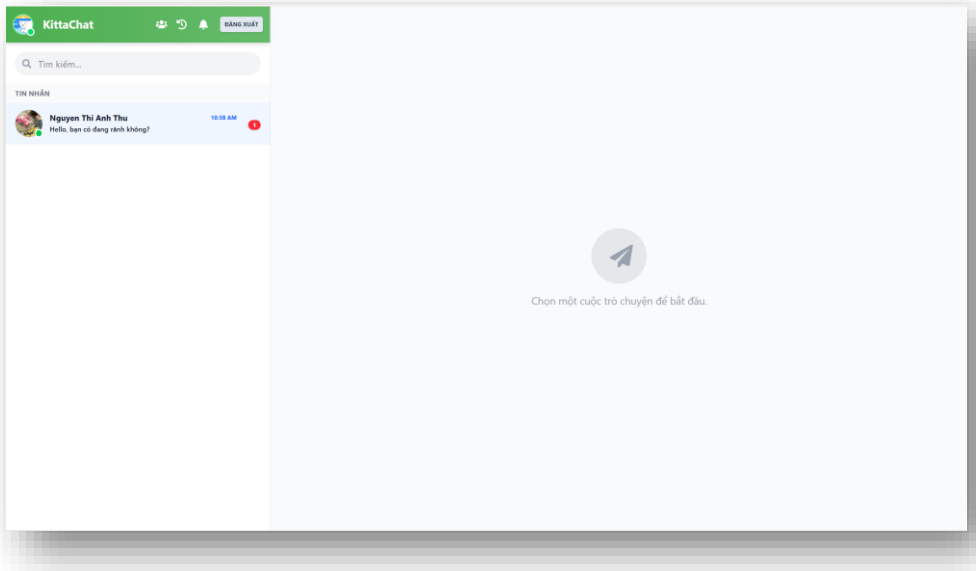

Figure 24 *The recipient's chat interface when the message hasn't been read*

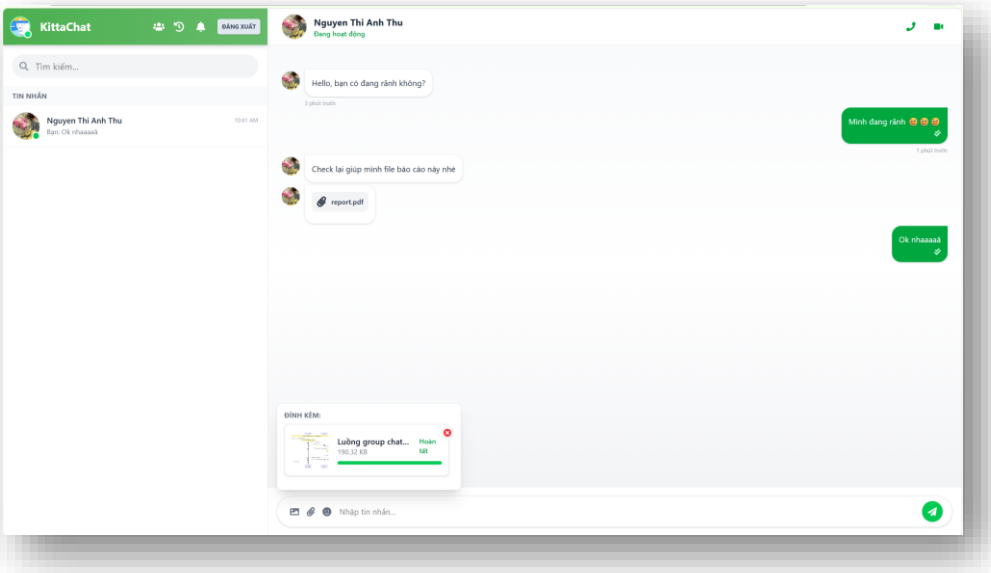

Figure 25 *UI Chat with text, emoji, file and Picture preview*

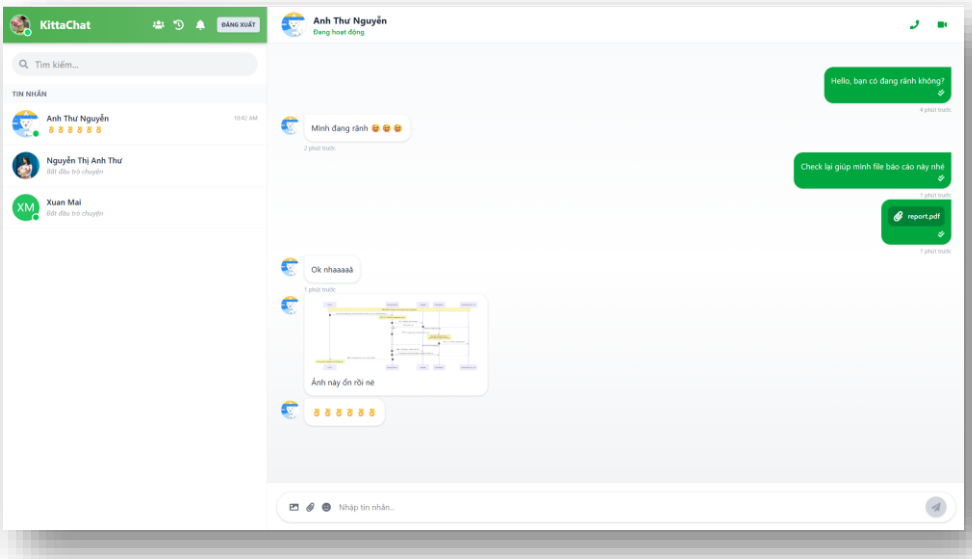

Figure 26 *UI Chat when the message is marked as Read*

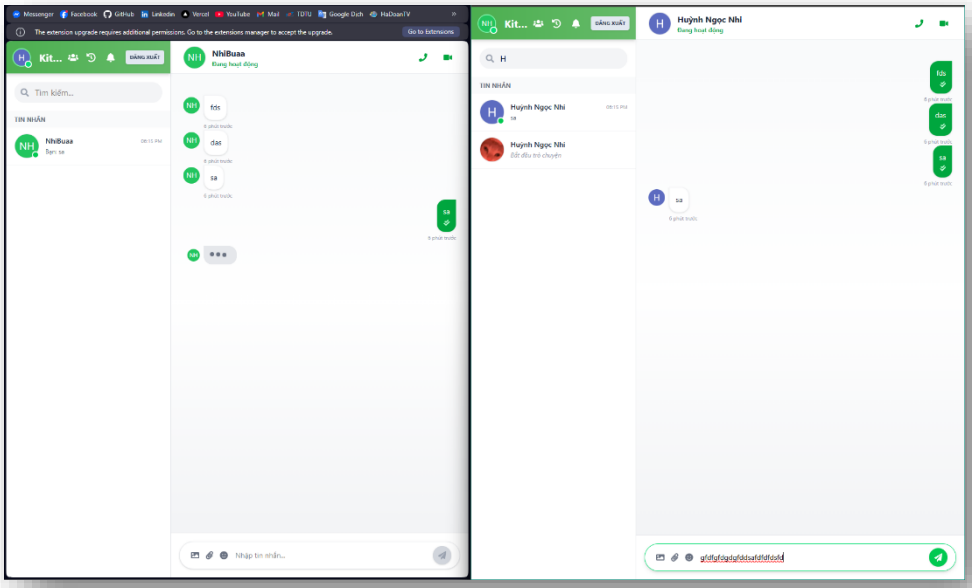

Figure 27 *UI Chat display typing status*

4.5.7 Audio and Video Call Interface

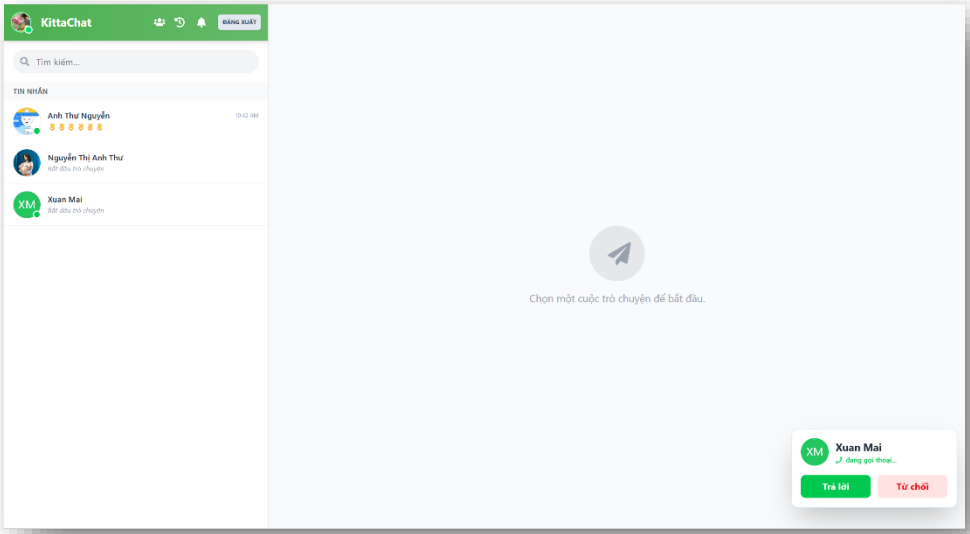

Figure 28 *UI Incoming Call*

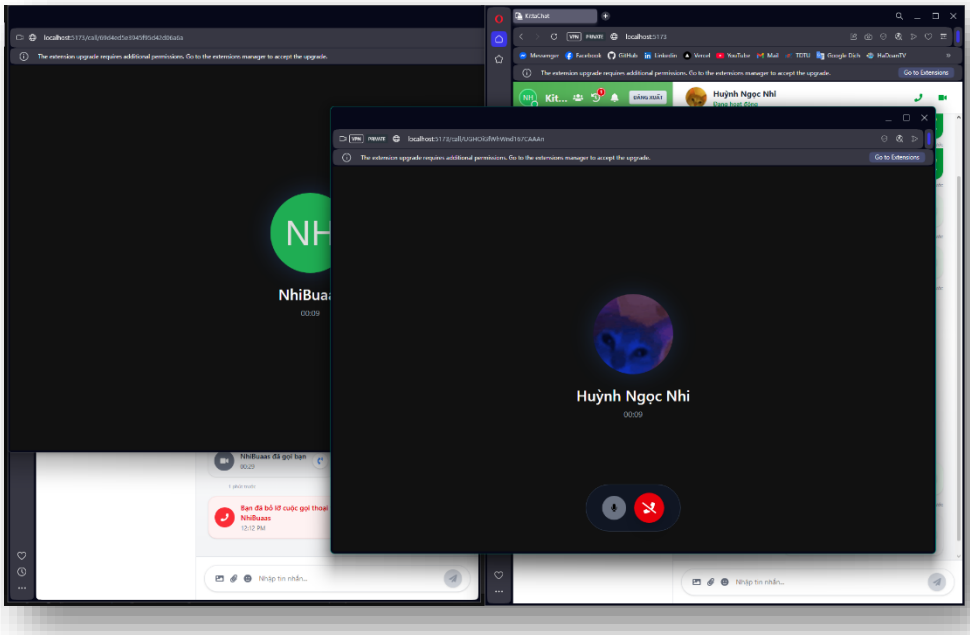

Figure 29 *UI During an Audio Call*

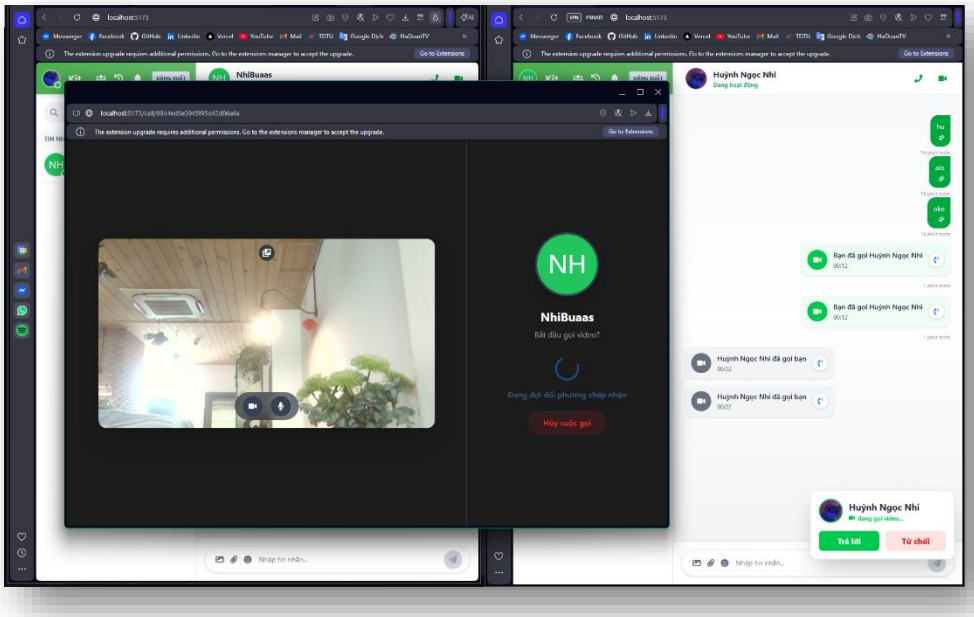

Figure 30 *UI Start a Video Call*

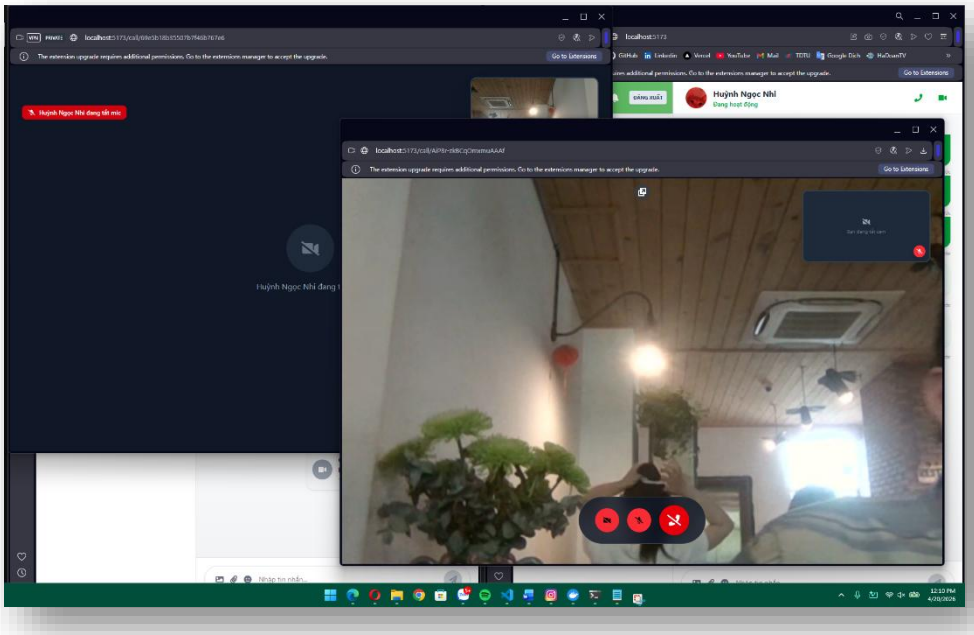

Figure 31 *UI During a Video Call*

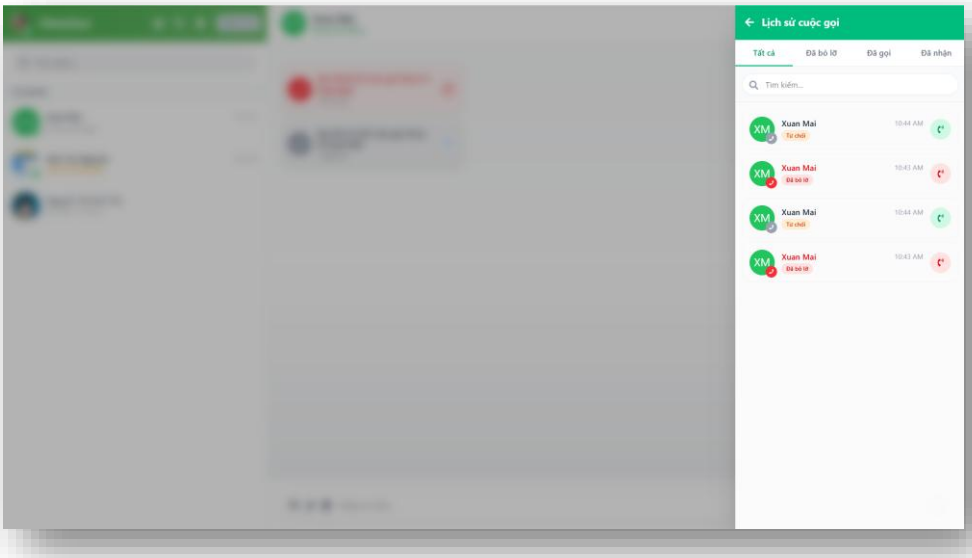

Figure 32 *UI History Call*

4.5.8 Forgot and Reset Password Interface

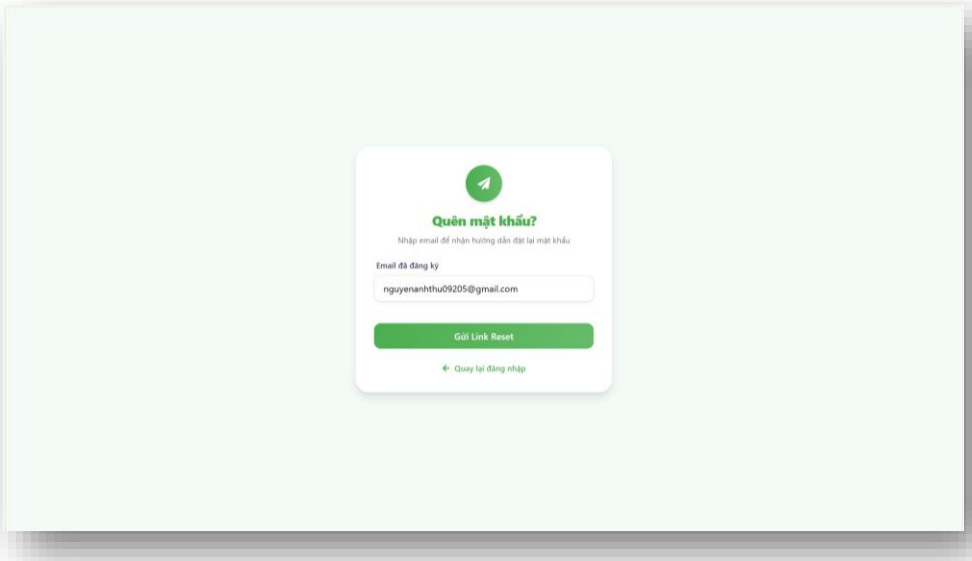

Figure 33 *UI Forgot Password*

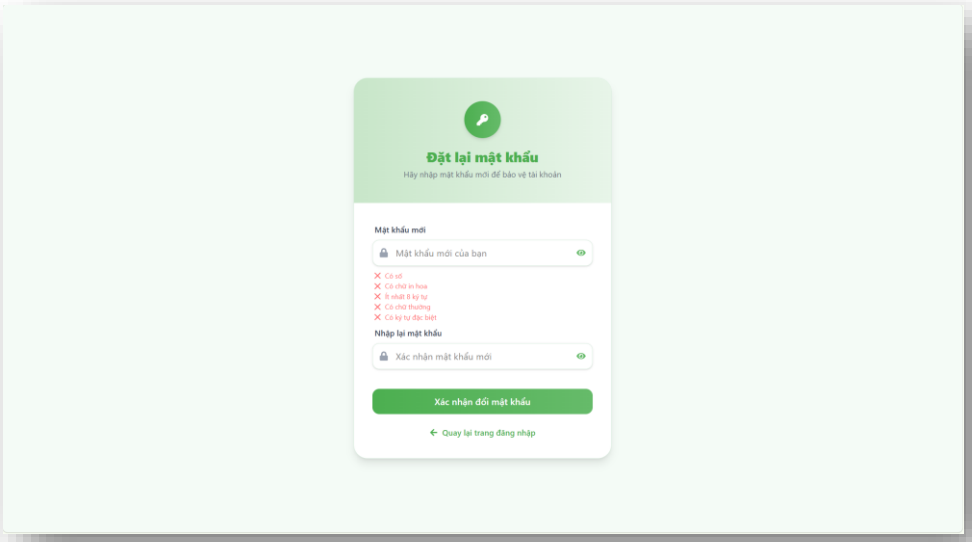

Figure 34 *UI Reset Password*

4.5.9 Group Chat Interface

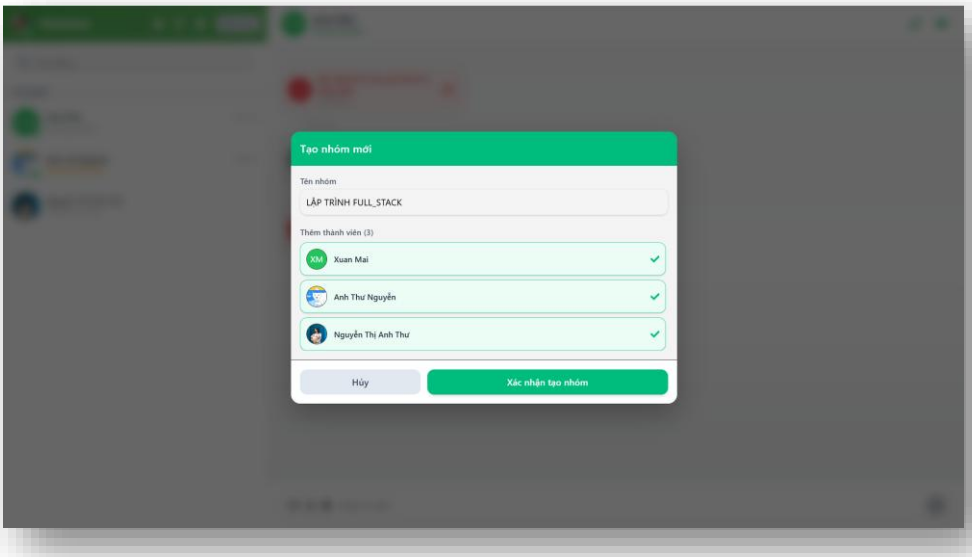

Figure 35 *UI Create Group Chat*

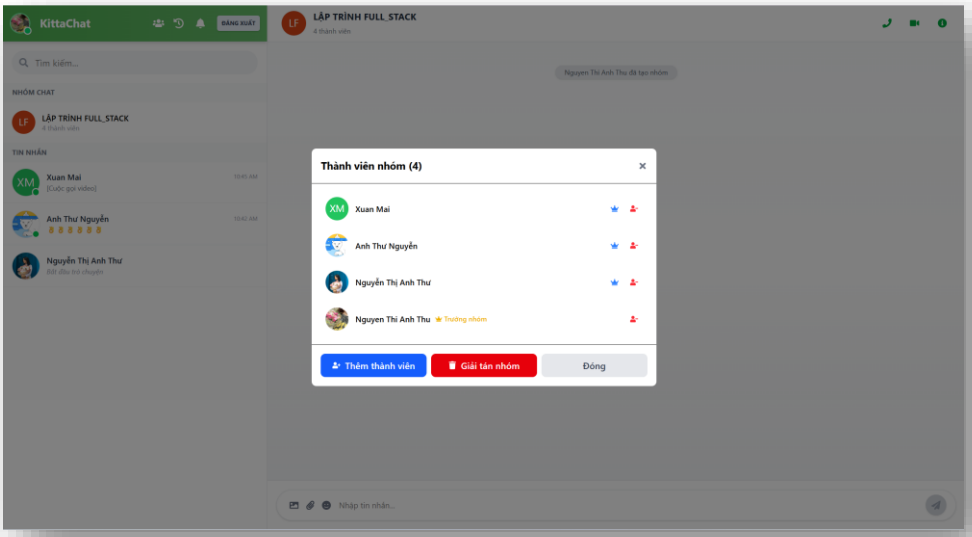

Figure 36 *UI Member Management Interface*

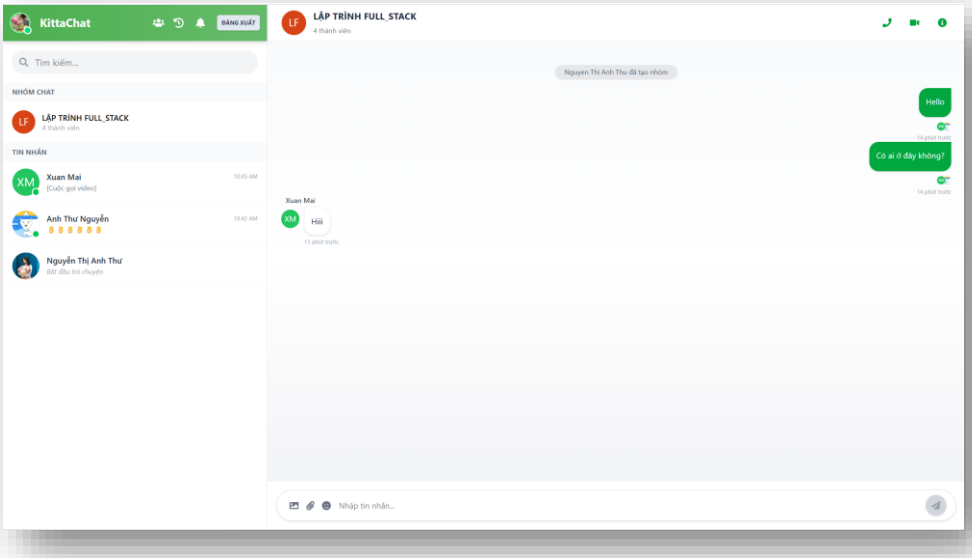

Figure 37 *UI Group Chat when*

4.5.10 Manage Profile Interface

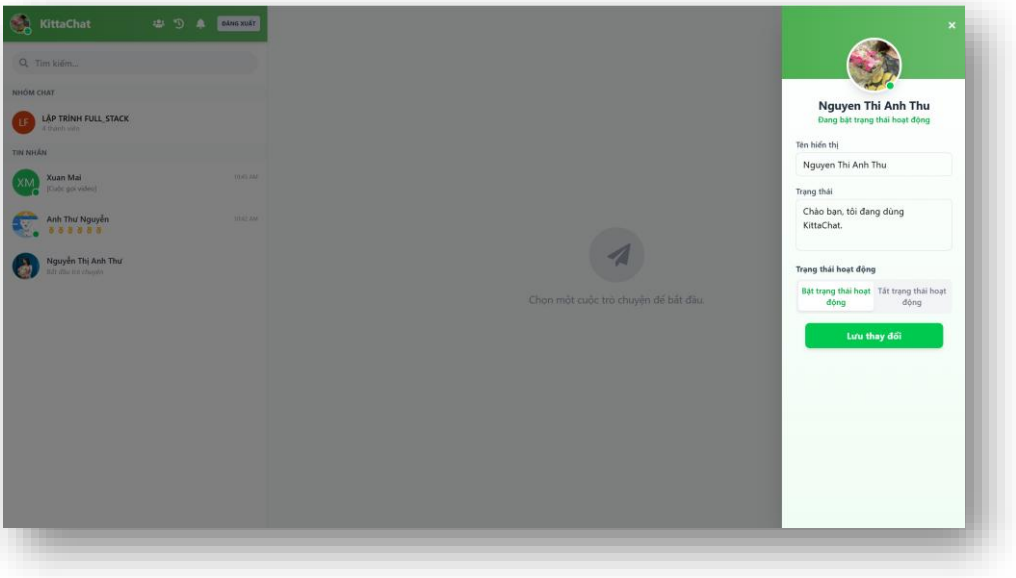

Figure 38 *UI Profile Management*

4.6 API Design

This section presents the RESTful APIs of the **KittaChat** system, supporting user management, authentication, messaging, and group communication via standard HTTP methods.

Category	Function	Method	URL	Description
AUTHENTICATION	Register	POST	/api/auth/register	Register a new account using email and password
	Login	POST	/api/auth/login	Login using email and password
	Google Login	POST	/api/auth/google	Login using Google account
	Forgot Password	POST	/api/auth/forgot-password	Send password reset email
	Reset Password	POST	/api/auth/reset-password/:id/:token	Reset password using token

USER MANAGEMENT	Get Profile	GET	/api/users/profile	Retrieve current user profile information
	Update Profile	PUT	/api/users/profile	Update user profile (avatar, display name)
	Get User By ID	GET	/api/users/:id	Retrieve user information by ID
	Get All Users	GET	/api/users	Retrieve all users
	Search Users	GET	/api/users/search	Search for users
FRIEND MANAGEMENT	Get Friends	GET	/api/users/friends	Retrieve friend list
	Get Online Friends	GET	/api/users/online-friends	Retrieve online friends
	Get Friend Requests	GET	/api/users/friend-requests	Retrieve friend requests
	Send Friend Request	POST	/api/users/friend-request	Send a friend request
	Accept Friend Request	POST	/api/users/accept-friend	Accept a friend request
	Reject Friend Request	POST	/api/users/reject-friend	Reject a friend request
	Get Sidebar List	GET	/api/users/sidebar-list	Retrieve sidebar user list
MESSAGING	Create Message	POST	/api/messages	Create a new message
	Get Messages	GET	/api/messages/:userId1/:userId2	Retrieve messages between two users
	Sync Missed Messages	GET	/api/messages/sync	Synchronize missed messages
GROUPS	Create Group	POST	/api/groups	Create a new group
	Get My Groups	GET	/api/groups	Retrieve user groups

	Get Group By ID	GET	/api/groups/:groupId	Retrieve group information by ID
	Add Member	POST	/api/groups/:groupId/add-member	Add member to group
	Remove Member	POST	/api/groups/:groupId/remove-member	Remove member from group
	Transfer Admin	POST	/api/groups/:groupId/transfer-admin	Transfer group admin role
	Rename Group	PUT	/api/groups/:groupId/rename	Rename group
	Delete Group	DELETE	/api/groups/:groupId	Delete group
CALL	Get Call History	GET	/api/calls/history	Retrieve call history
	Get Missed Calls	GET	/api/calls/missed	Retrieve missed calls
	Mark Call Read	POST	/api/calls/:id/read	Mark call as read
	Mark All Calls Read	POST	/api/calls/read-all	Mark all calls as read
FILE MANAGEMENT	Init Upload	POST	/api/files/init	Initialize file upload
	Get Presigned URL	POST	/api/files/get-presigned-url	Get presigned URL for S3 upload
	Complete Upload	POST	/api/files/complete	Complete file upload

	Upload Single File	POST	/api/files/upload-single	Upload a single file
SYSTEM	Health Check	GET	/healthz	Check server health (database connection)
	Ready Check	GET	/readyz	Check server readiness

Table 3 *API design*

4.7 High Performance & Scalability

4.7.1 Load Balancing with Nginx

Nginx is configured as a reverse proxy and load balancer to distribute incoming client requests across multiple backend containers running in parallel. This setup enables horizontal scaling and improves the system's ability to handle high concurrency.

To support load balancing, an upstream cluster is defined in the Nginx configuration. This cluster represents a group of backend instances, allowing Nginx to dynamically route incoming requests.

In this configuration, the **least connection strategy** is applied, which forwards each request to the backend instance with the fewest active connections. This helps evenly distribute traffic and prevents any single server from becoming overloaded.

Additionally, failure handling is configured using parameters such as max_fails and fail_timeout, allowing Nginx to temporarily exclude unhealthy backend instances. The keepalive setting is also enabled to maintain persistent connections between Nginx and backend servers, reducing connection overhead and improving performance.

This configuration works together with Docker-based scaling, where multiple backend containers are deployed simultaneously, enabling efficient load distribution across instances.

```
upstream backend_cluster {
    least_conn;

    server backend:3000 max_fails=3 fail_timeout=30s;
    keepalive 64;
}
```

Figure 39 *Nginx Upstream Load Balancing Configuration*

In addition to load balancing, Nginx is configured to act as a reverse proxy for REST API requests.

```
location /api/ {
    limit_req zone=api_limit burst=20 nodelay;

    proxy_pass         http://backend_cluster;
    proxy_http_version 1.1;
    # BẬT keepalive - reuse connection pool
    proxy_set_header   Connection "";
    proxy_set_header   Host              $host;
    proxy_set_header   X-Real-IP         $remote_addr;
    proxy_set_header   X-Forwarded-For   $proxy_add_x_forwarded_for;
    proxy_set_header   X-Forwarded-Proto $scheme;
    # Forward Origin để Express CORS kiểm tra được origin thật
    proxy_set_header   Accept            $http_origin;

    proxy_connect_timeout 60s;
    proxy_send_timeout    60s;
    proxy_read_timeout    60s;

    proxy_buffering    on;
    proxy_buffer_size  4k;
    proxy_buffers      8 4k;
}
```

Figure 40 *Nginx API Reverse Proxy Configuration*

This configuration ensures efficient routing of API requests to backend instances through the upstream cluster.

Key aspects of this configuration include:

- The proxy_pass directive routes requests to the backend instances.
- The limit_req setting applies rate limiting to prevent request flooding and improve system stability.

- Headers such as X-Forwarded-For and X-Real-IP are preserved to maintain client identity and support logging.
- The Connection “” setting enables keep-alive connections, reducing overhead and improving performance.
- Timeout configurations ensure that long-running requests are handled reliably without premature termination.
- Buffering is enabled to improve response handling efficiency.

To support real-time communication, Nginx is configured to properly handle WebSocket connections used by Socket.IO.

```
location /socket.io/ {
    limit_conn conn_limit 10;

    proxy_pass         http://backend_cluster;
    proxy_http_version 1.1;

    # WebSocket upgrade headers - giữ nguyên "upgrade"
    # KHÔNG set Connection "" vì WS cần upgrade
    proxy_set_header   Upgrade    $http_upgrade;
    proxy_set_header   Connection $connection_upgrade;
    proxy_set_header   Host       $host;
    proxy_set_header   X-Real-IP  $remote_addr;
    proxy_set_header   X-Forwarded-For $proxy_add_x_forwarded_for;

    # WebSocket timeout phải LỚN - Socket.IO ping/pong ~20s interval
    proxy_read_timeout  86400s;
    proxy_send_timeout  86400s;

    # TẮT buffering cho WebSocket - CRITICAL
    proxy_buffering off;
}
```

Figure 41 *Nginx WebSocket Proxy Configuration (Socket.IO)*

This configuration ensures that WebSocket connections are correctly upgraded and maintained between clients and backend servers.

- The Upgrade and Connection headers are required to establish and maintain WebSocket connections.
- The proxy_http_version 1.1 setting is necessary for WebSocket support.

- Long timeout values are configured to prevent connection drops, as real-time communication requires persistent connections.
- Buffering is disabled to allow immediate data transmission without delay, which is critical for real-time messaging.
- The limit_conn directive helps prevent connection flooding by limiting the number of simultaneous connections per client.

This setup is essential for supporting real-time communication in a scalable environment.

Finally, this Nginx configuration is integrated with Docker-based deployment, where multiple backend instances are deployed using container replication. This allows the system to run multiple backend containers in parallel, forming a scalable backend cluster.

```
backend:
  build:
    context: ./server
    dockerfile: Dockerfile
    target: prod
  deploy:
    replicas: 3
```

Figure 42 *Docker Compose Configuration with Multiple Backend Replicas*

4.7.2 Scaling WebSocket with Redis Pub/Sub

In a distributed system with multiple backend instances, WebSocket connections may be handled by different servers. Without a synchronization mechanism, real-time events (such as messages or call signals) would not be delivered consistently across all connected clients.

To solve this problem, the system uses Redis Pub/Sub in combination with the Socket.IO Redis adapter to synchronize events across all backend instances.

```
const redisUrl = process.env.REDIS_URL || "redis://redis:6379";
const pubClient = createClient({ url: redisUrl });
const subClient = pubClient.duplicate();

pubClient.on("error", (err) => console.error("[Redis PubClient] Error:", err));
subClient.on("error", (err) => console.error("[Redis SubClient] Error:", err));

Promise.all([pubClient.connect(), subClient.connect()])
    .then(() => {
        io.adapter(createAdapter(pubClient, subClient));
        console.log("[Socket] Redis adapter connected");
    })
    .catch((err) => {
        // FAIL FAST: Redis là core dependency, không chạy nếu không có Redis
        console.error(`[Socket] FATAL: Redis connection failed: ${err.message}`);
        process.exit(1);
    });

app.set("socketio", io);
app.set("redisClient", pubClient);
io.redisClient = pubClient;
```

Figure 43 *Redis Adapter Configuration*

In this configuration, two Redis clients are created

- A **publisher client (pubClient)** is responsible for publishing events to Redis.
- A **subscriber client (subClient)** listens for events from Redis.

These clients are then connected to the Socket.IO server using the Redis adapter

- The createAdapter(pubClient, subClient) function allows all backend instances to share real-time events.
- When a message is emitted on one server, it is published to Redis and then distributed to all other servers.
- This ensures that users connected to different backend containers can still communicate seamlessly.

This approach removes the limitations of single-instance WebSocket communication and enables horizontal scaling, ensuring consistent real-time message delivery across distributed backend systems.

This Redis-based synchronization mechanism works together with Nginx load balancing. While Nginx distributes incoming client connections across multiple

backend instances, Redis Pub/Sub ensures that all real-time events are shared and synchronized between these instances.

As a result, even if two users are connected to different backend servers, messages and events are still delivered correctly through Redis. This architecture decouples connection handling from event propagation, enabling scalable and reliable real-time communication.

4.7.3 Evidence

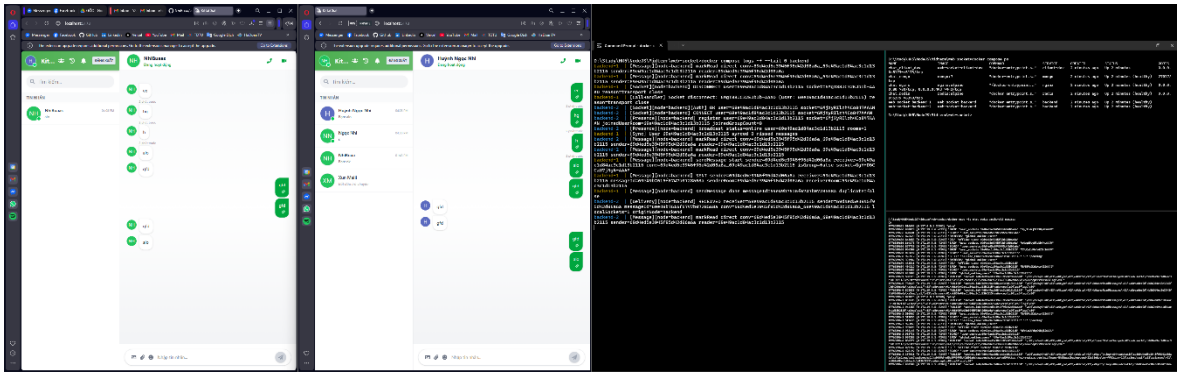

Figure 44 *Logs two backend containers handling real-time messages*

The following logs demonstrate real-time communication between two clients, along with system logs and container status, demonstrating distributed message processing.

On the left side, two separate client interfaces are shown exchanging messages in real time. On the right side, Docker logs indicate that multiple backend containers (e.g., backend-1 and backend-2) are running simultaneously.

When a message is sent from one client, it is processed by one backend instance, as indicated by the log entries. The event is then propagated through Redis Pub/Sub, allowing another backend instance to receive the same message and deliver it to the target client.

The logs clearly show that different backend containers are involved in handling the same communication flow, confirming that:

- Nginx distributes connections across multiple backend instances.

- Redis Pub/Sub synchronizes real-time events between servers.
- Clients connected to different backend nodes can communicate seamlessly.

CHAPTER 5. SYSTEM DEPLOYMENT

5.1 Development Environment and Tools

The KittaChat system is developed using a modern web development environment to ensure efficiency and maintainability. Visual Studio Code (VS Code) is used as the primary code editor for both frontend and backend development due to its lightweight performance and extensive plugin support.

GitLab is used as the version control system to manage source code, track changes, and support collaboration among team members. GitLab Issues is also utilized for tracking bugs and managing development tasks.

Docker and Docker Compose are used to containerize and manage system services, including backend, database, Redis, and Nginx. This approach ensures consistency across different development environments and simplifies deployment.

Google Chrome is used for testing, debugging, and monitoring real-time communication via WebSocket.

5.2 Programming Languages and Frameworks

The system is built using JavaScript as the primary programming language. The backend is developed with Node.js and Express.js, providing a lightweight and efficient environment for handling REST APIs and real-time communication.

Socket.IO is integrated to enable WebSocket-based real-time messaging between clients and the server. WebRTC is used to support peer-to-peer audio and video communication.

On the frontend, ReactJS is used to build a dynamic and responsive user interface. Vite is utilized as the build tool to provide fast development and efficient module bundling. Tailwind CSS is applied for styling, allowing rapid UI development with a clean and modern design.

5.3 Source Code Structure

5.3.1 Overall Project Structure

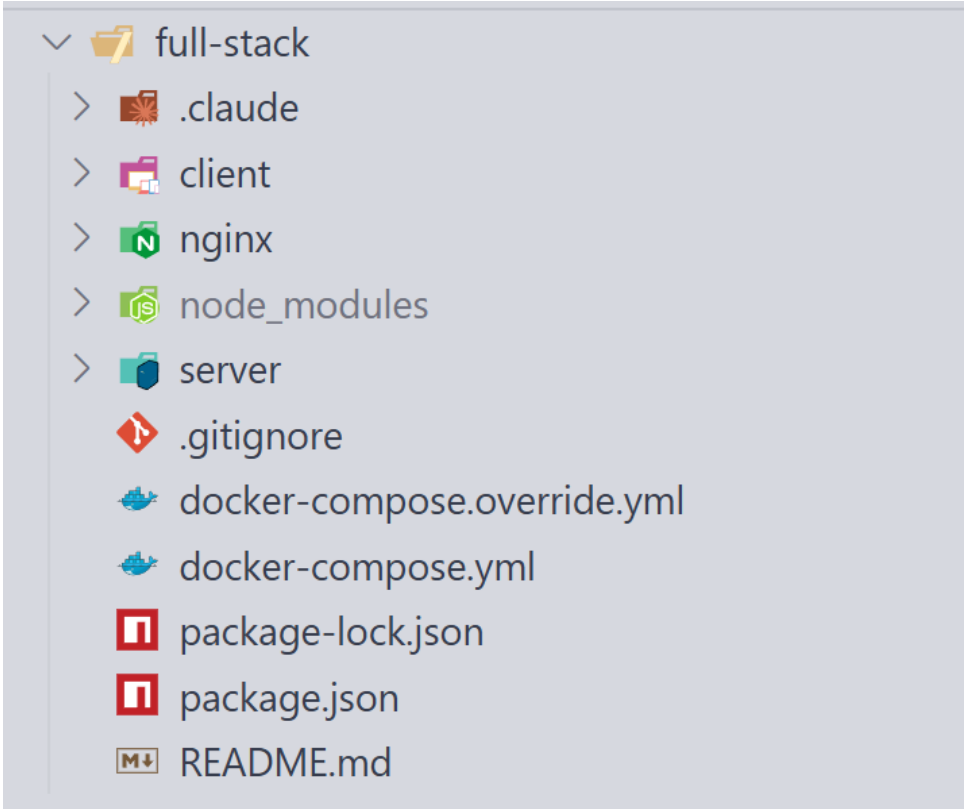

Figure 45 *Overall Project Structure*

The overall project structure of the KittaChat system is organized following a **client-server architecture with a modular monolithic backend design**, as shown above. The project is divided into three main components: frontend, backend, and infrastructure configuration.

The **client** directory contains the frontend application developed using ReactJS. It is responsible for rendering the user interface, handling user interactions, and communicating with the backend through REST APIs and WebSocket (Socket.IO).

The **server** directory implements the backend system using Node.js and Express.js. It handles business logic, user authentication, real-time communication, and interaction with the database and Redis.

The **nginx** directory includes configuration files for Nginx, which acts as a reverse proxy and load balancer to distribute incoming requests across multiple backend instances.

The system uses Docker Compose (**docker-compose.yml** and **docker-compose.override.yml**) to orchestrate and manage all services, including frontend, backend, database, Redis, and Nginx. This setup enables consistent development environments and supports **horizontal scalability**.

Additional files such as **package.json**, **package-lock.json**, and **README.md** are used for dependency management and project documentation.

Overall, this structure ensures a clear separation of concerns and supports scalable deployment of a real-time communication system.

5.3.2 Frontend (Client) Source Code Structure

The frontend of the KittaChat system is developed using ReactJS and follows a **modular, component-based architecture** to ensure maintainability and scalability. The client application is organized into multiple directories, each responsible for a specific aspect of the system.

The **src** directory contains the core application logic and is structured into several submodules. The **components** folder includes reusable UI elements such as chat interfaces, message items, and call components. The **pages** directory defines the main application views, including authentication screens and chat pages. The **context** folder is used for global state management, while the **hooks** directory contains custom React hooks for handling reusable logic such as WebSocket communication and user interactions.

The **services** directory manages communication with the backend, including REST API calls and Socket.IO integration for real-time features. The **utils** folder provides helper functions used throughout the application.

Core files such as **App.jsx** define the main application structure, while **main.jsx** serves as the entry point of the React application. The **firebase.js** file is used to configure Firebase Authentication for Google Sign-In functionality.

The frontend also integrates **WebRTC** for peer-to-peer audio and video communication between users.

In addition, configuration files such as **vite.config.js**, **eslint.config.js**, and Docker-related files are included to support development, code quality, and deployment processes.

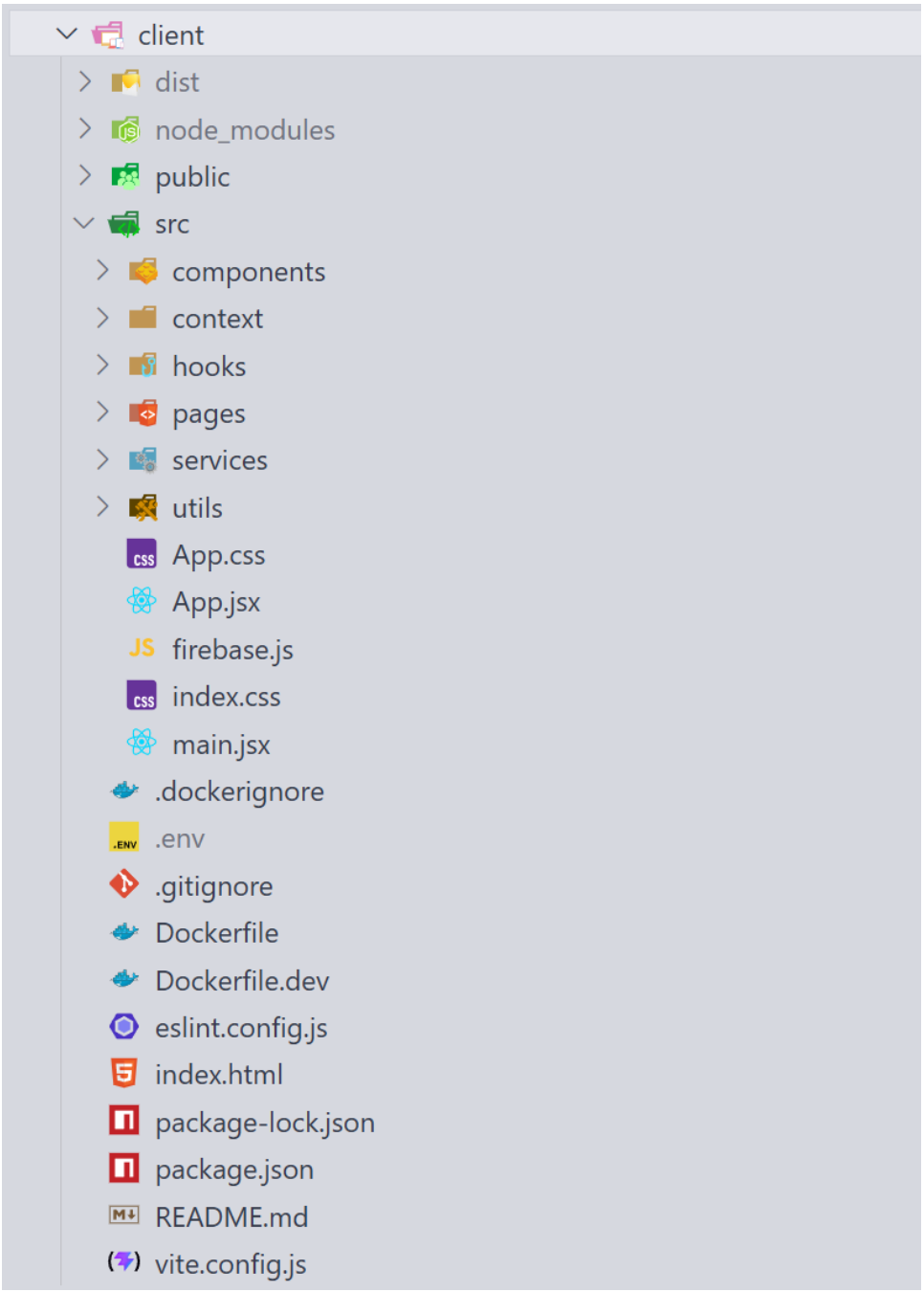

Figure 46 *Frontend (Client) Source Code Structure*

5.3.3 Backend (Server) Source Code Structure

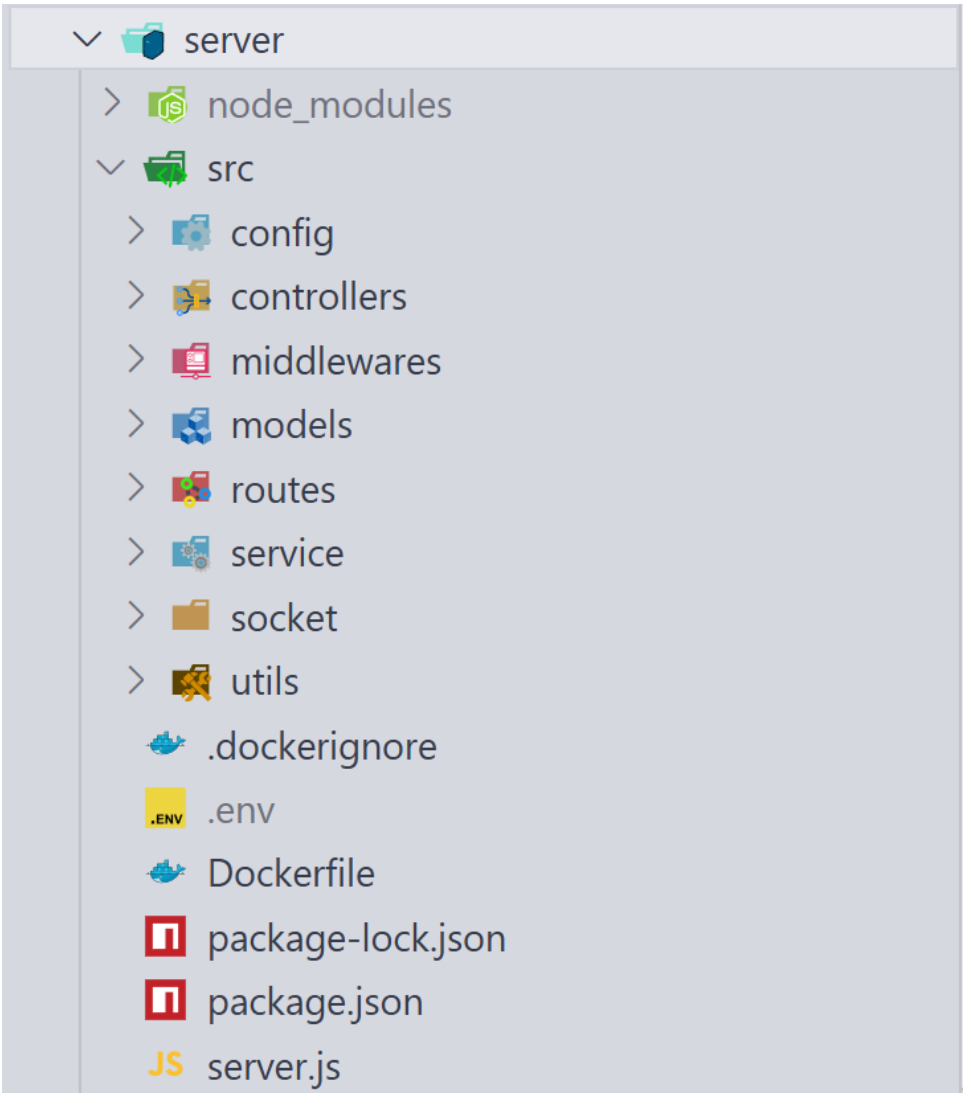

Figure 47 *Backend (Server) Source Code Structure*

The backend of the KittaChat system is developed using Node.js and Express.js, following a modular and layered architecture to ensure maintainability, scalability, and clear separation of concerns. The server-side application is organized into multiple directories, each responsible for a specific functionality.

The **src** directory contains the core backend logic and is divided into several modules. The **controllers** folder handles incoming HTTP requests and returns appropriate responses. The **routes** directory defines API endpoints and maps them to

corresponding controllers. The **models** folder defines MongoDB schemas using Mongoose for entities such as users, messages, groups, and call histories.

The **service** layer contains business logic and core functionalities, separating application logic from request handling. The **middlewares** directory includes authentication (JWT), validation, and error-handling logic.

The **config** directory is responsible for managing system configurations and integrating external services required by the backend. It handles the initialization of authentication services and establishes connections to external systems such as Redis for caching and real-time message synchronization, ensuring consistent configuration and efficient resource management across the application.

The **socket** directory implements real-time communication using Socket.IO, including message handling, user presence updates, and signaling for WebRTC-based audio and video calls. The **utils** folder provides helper functions used across the backend.

The main entry point of the backend is **server.js**, which initializes the Express application and sets up WebSocket connections. Additional configuration files such as **Dockerfile** and **.env** are used for containerization and environment management.

5.4 Database Deployment

MongoDB is used as the primary database system for storing user data, chat messages, group information, and call histories. During development, MongoDB is deployed using Docker containers to ensure a consistent and isolated environment.

The system is designed with efficient data models and indexing strategies to support fast data retrieval and scalability. The use of **conversationId** for message storage enables efficient querying and pagination of chat history.

In addition, **indexing strategies such as compound indexes on conversationId and timestamps** are applied to optimize query performance for large-scale message data.

CHAPTER 6. DEVELOPMENT AND EVALUATION

6.1 Testing Strategy

The testing strategy of the KittaChat system is designed to ensure that all components function correctly and meet the specified requirements. Multiple levels of testing are applied, including unit testing, integration testing, and system testing.

- **Unit Testing** is conducted to verify individual components of the system in isolation. This includes testing backend modules such as authentication, message processing, and API endpoints, as well as frontend components like chat interfaces and user interactions. Each unit is tested independently to ensure correctness before integration.
- **Integration Testing** is performed to ensure that different components of the system work together seamlessly. This includes verifying communication between the frontend and backend through REST APIs and WebSocket (Socket.IO), as well as ensuring proper interaction with MongoDB and Redis. Particular attention is given to real-time message synchronization and event handling across services.
- **System Testing** is carried out to evaluate the complete functionality of the application as a whole. End-to-end scenarios such as user registration, friend management, real-time messaging, group chat, and audio/video calling are tested to ensure that the system behaves as expected. This level of testing confirms that all integrated components operate correctly in a real-world usage environment.

6.2 Performance Testing

Performance testing is conducted to evaluate the responsiveness and stability of the system under different conditions. The system demonstrates low latency in real-time messaging due to the use of WebSocket (Socket.IO), enabling instant message delivery without page reload.

Basic testing shows that the system can handle multiple concurrent users with stable response times. Message delivery typically occurs within a short time frame, and WebRTC-based audio/video calls maintain acceptable quality under normal network conditions.

6.3 Bug Tracking and Issue Management

Bugs and issues encountered during development are managed using Git-based version control systems and issue tracking tools such as GitLab Issues. Each issue is documented, tracked, and resolved systematically to ensure system stability.

Common issues include connection interruptions, message synchronization delays, and UI inconsistencies, which are addressed through debugging and iterative improvements. GitLab is also used to manage commits, track changes, and coordinate development among team members.

6.4 Performance and Scalability Testing

To evaluate system scalability, the application is tested in a distributed environment with multiple backend instances. Redis Pub/Sub is used to synchronize messages across servers, ensuring that users connected to different instances can communicate seamlessly.

Load testing is conducted to simulate normal usage scenarios with multiple concurrent users sending messages simultaneously. The system maintains stable performance with minimal latency.

Stress testing is performed to observe system behavior under high load conditions. Results show that while performance may degrade slightly under extreme conditions, the system remains functional without crashing.

Endurance testing is conducted by running the system continuously over a period of time to evaluate stability. The system maintains consistent performance and does not exhibit memory leaks or significant degradation.

Overall, the results indicate that the system is capable of handling real-time communication efficiently and can scale horizontally using Redis and Nginx.

6.5 Test Account

Email	Password
nguyenanhthu09205@gmail.com	@Anhthu123
huynhngocnhi@gmail.com	@Nghi123
gohuynh1123@gmail.com	@Nnhi123

Table 4 *Test Account*

CHAPTER 7. CONCLUSION

7.1 Achieved Results

The KittaChat system has been successfully developed as a real-time communication platform that fulfills the requirements from Level 1 to Level 4 as defined in the project specification.

7.1.1 Level 1 – Basic Chat System

The system successfully implements core real-time communication features. Users can register and log in using JWT-based authentication or Google Sign-In via Firebase. The application supports user search and friend management, including sending, accepting, and rejecting friend requests.

In addition, the system provides one-to-one private chat with real-time message delivery using WebSocket (Socket.IO). Messages are stored in MongoDB and can be retrieved when reopening conversations. The system also supports message status tracking such as “sent” and “seen”.

7.1.2 Level 2 – Group Chat & Multimedia

The system extends its functionality by supporting group chat features. Users can create groups, add members, leave groups, and administrators can remove members or delete groups. Ownership transfer is also implemented to ensure proper group management.

Multimedia messaging is supported, allowing users to send images, files, and emojis. Image preview functionality is also implemented to enhance user experience.

Furthermore, the system includes real-time presence features such as online/offline status and typing indicators, which improve interaction and responsiveness during conversations.

7.1.3 Level 3 – Audio/Video Calling (WebRTC)

The system integrates WebRTC to support peer-to-peer audio and video communication between users. A signaling mechanism is implemented using Socket.IO to exchange session descriptions and ICE candidates.

Users can initiate calls, accept or reject incoming calls, and control ongoing calls through features such as microphone and camera toggling, as well as ending calls. The system also provides an incoming call notification interface to enhance usability.

7.1.4 Level 4 – High Performance & Scalability

To ensure the system can handle multiple concurrent users, scalability solutions have been implemented. The backend is deployed with multiple instances, and Redis Pub/Sub is used as a message broker to synchronize real-time events across servers.

Nginx is configured as a reverse proxy and load balancer to distribute incoming traffic efficiently. In addition, Redis is also used for caching frequently accessed data such as user presence and recent messages, reducing database load and improving system performance.

These optimizations demonstrate that the system can maintain stability and responsiveness even in distributed environments with multiple server instances.

7.2 Advantages and Limitations

7.2.1 Advantages

- **User-friendly Interface**

  The system is designed with a simple, intuitive, and responsive user interface using ReactJS. Users can easily perform core actions such as authentication, friend management, real-time messaging, and initiating audio/video calls without requiring technical knowledge. The interface

dynamically updates based on real-time events, ensuring smooth interactions and providing a modern, user-friendly experience.

❖ **Complete Real-time Communication Features**

The system successfully implements a comprehensive set of real-time communication features, including private chat, group chat, multimedia messaging, emoji support, and audio/video calling. All interactions are processed instantly without requiring page reload, providing a seamless user experience comparable to modern messaging platforms.

❖ **Efficient Real-time Messaging with WebSocket**

By utilizing WebSocket (Socket.IO), the system enables bidirectional communication between clients and the server. Messages, typing indicators, read receipts, and online status updates are transmitted instantly, significantly reducing latency and improving responsiveness.

❖ **Peer-to-Peer Video Call with WebRTC**

The integration of WebRTC enables direct peer-to-peer communication for audio and video calls. This approach reduces server load and ensures low-latency media transmission. The signaling mechanism implemented via Socket.IO allows reliable connection establishment between users.

❖ **Scalable Architecture with Redis and Nginx**

The system is designed to support high concurrency through horizontal scaling. Redis Pub/Sub is used to synchronize real-time data across multiple backend instances, while Nginx acts as a reverse proxy and load balancer to distribute incoming traffic efficiently. This ensures stable performance even in distributed environments.

❖ **Optimized Data Handling and Message Storage**

MongoDB is used as the primary database to efficiently store user data, message logs, group information, call histories, and file metadata. The use of conversationId and optimized indexing strategies enables fast message retrieval and efficient pagination. Additionally, an idempotency mechanism is

implemented to prevent duplicate messages during network retries, ensuring data consistency.

❖ **Flexible Authentication Mechanism**

The system supports both JWT-based authentication and Google Sign-In via Firebase. This provides flexibility for users while ensuring secure access control through token-based session management.

❖ **Modular and Containerized Deployment**

The application is structured using a modular architecture and deployed with Docker and Docker Compose. This approach simplifies deployment, ensures consistency across different environments, and enhances system scalability and maintainability.

7.2.2 Limitations

❖ **Limited Advanced Security Features**

Although the system implements basic authentication using JWT and Google Sign-In, advanced security mechanisms such as end-to-end encryption (E2EE) have not been implemented. This may limit data privacy and security in sensitive communication scenarios.

❖ **Lack of Group Audio/Video Calling**

The current system only supports one-to-one audio and video calls using WebRTC. Group calling or multi-user conferencing features have not yet been implemented, which limits the system's capabilities compared to fully developed communication platforms.

❖ **Limited Real-world Scalability Testing**

While the system is designed with scalability in mind using Redis Pub/Sub and Nginx load balancing, testing has primarily been conducted in a controlled or simulated environment. The system has not yet been evaluated under real-world conditions with a large number of concurrent users.

- **Dependence on Network Conditions**

  The performance of real-time messaging and WebRTC-based audio/video calls depends heavily on network quality. Poor network conditions may lead to increased latency, packet loss, or unstable connections.

- **Limited Feature Set Compared to Commercial Platforms**

  Some advanced features commonly found in modern messaging platforms, such as message search, push notifications, message reactions, voice message, and advanced media management, have not yet been fully implemented.

- **Basic Error Handling and Recovery**

  The current system includes basic error handling; however, more robust mechanisms for failure recovery, such as automatic reconnection strategies for WebRTC sessions or advanced retry mechanisms, can be further improved.

7.3 Future Development Directions

In the future, the KittaChat system can be enhanced across several dimensions to improve both functionality and performance. A primary objective is the implementation of End-to-End Encryption (E2EE) to ensure secure communication and protect user privacy. Additionally, the platform can be extended to support multi-party audio and video conferencing by leveraging advanced WebRTC architectures, such as Selective Forwarding Units (SFU), to efficiently handle group sessions.

From an infrastructure perspective, scalability can be further improved by migrating to cloud-native platforms such as AWS or Google Cloud, and by utilizing Kubernetes for container orchestration and distributed service management. Furthermore, to align with modern industry standards, features such as global message search, push notifications, message reactions, and real-time message editing can be integrated to enhance the overall user experience.

System performance can also be optimized through advanced Redis caching strategies and improved database indexing techniques. Finally, focusing on cross-platform compatibility, such as developing dedicated mobile applications, and

implementing more robust error-handling mechanisms will help ensure that KittaChat evolves into a stable, user-friendly, and production-ready system.

REFERENCES

[1] Authentication with Google

[2]WebSocket and Its Difference from HTTP

[2] What is WebSocket - How it Works - An Introduction

[3] WebRTC API - Web APIs | MDN

[4] Socket.IO